Distributed Systems

Simple Introduction Guide

Dr. Gehad Taher

Contents

Part One: Effective DS

1. Introduction to Distributed Systems

1.1 Introduction

Imagine you're using Google Maps to find the fastest route to a café. You type in the destination, and within seconds, your phone shows traffic updates, estimated arrival time, and alternate routes. Behind the scenes, dozens—maybe hundreds—of computers are working together to deliver that result. This is the magic of **distributed systems**.

Distributed systems are the invisible engines behind the digital experiences we rely on every day. From streaming a movie on Netflix to conducting a real-time financial transaction across continents, these systems quietly coordinate vast networks of machines to deliver seamless performance. As the world becomes increasingly data-driven, centralized computing models are reaching their limits. The sheer volume, velocity, and variety of data—generated by billions of devices and users—demand architectures that can scale, adapt, and recover gracefully from failure.

1.1.1 What Is a Distributed System?

A **distributed system** is a group of independent computers that work together to appear as a single system to the user Figure 1.1 shows that these computers:

- Communicate over a network
- Share resources and data
- Coordinate tasks to achieve a common goal

These systems are designed to handle massive workloads, scale across geographies, and remain resilient even when parts fail. They are the backbone of modern computing, enabling everything from cloud platforms and social media to scientific research and global commerce Figure 1.2 presents a powerful idea that says: We are living at the edge of a technological revolution. Fields like astronomy, biotechnology, sensor networks, and ubiquitous computing are generating unprecedented amounts of data and complexity. These breakthroughs are not isolated—they are deeply interconnected, and they all rely on distributed systems to function.

- **Astronomy:** Telescopes like JWST produce terabytes of data daily, requiring distributed storage and processing.
- **Gene Sequencing:** Modern labs analyze entire genomes using distributed computing pipelines.
- **Sensors:** Smart cities and industrial systems use distributed sensor networks for real-time monitoring.
- **Ubiquitous Computing:** Devices embedded in everyday life (phones, appliances, vehicles)

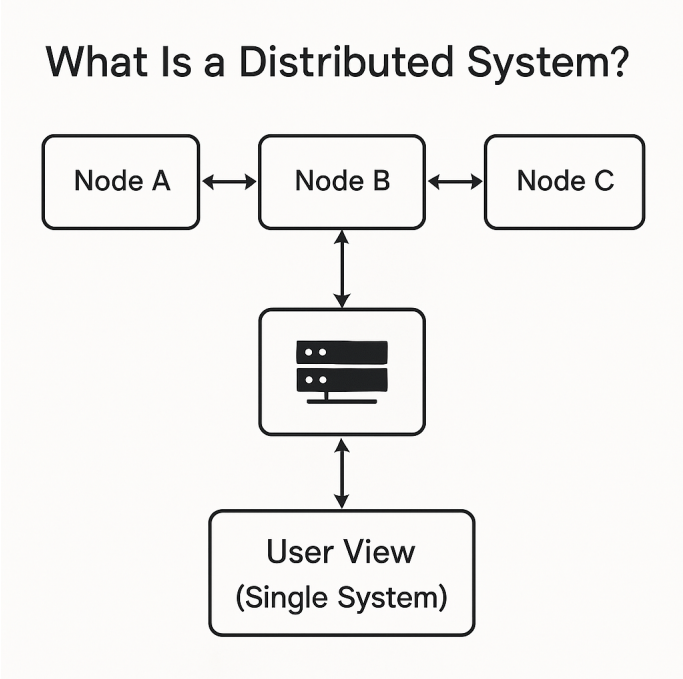


Figure 1.1: Distributed Systems Idea

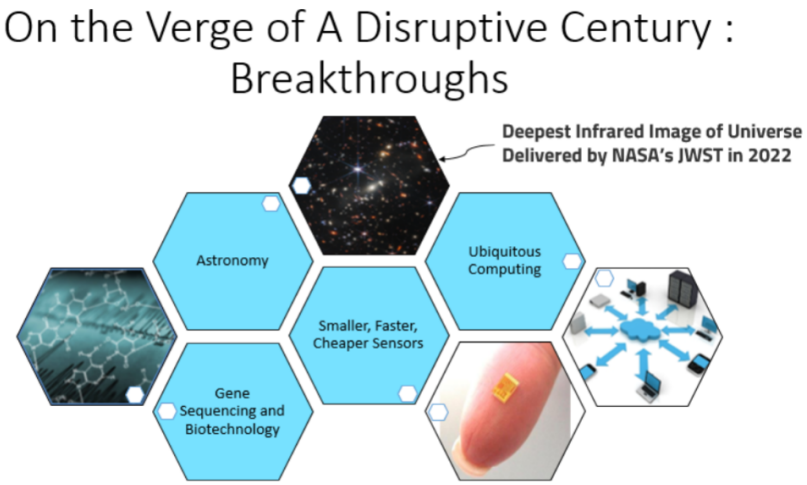


Figure 1.2: Foundations of Distributed systems

rely on distributed coordination.

1.1.2 We Live in a World of Data

We are surrounded by data—generated, transmitted, and consumed at staggering rates. Every click, swipe, transaction, and sensor reading contributes to a digital ecosystem that is growing exponentially. In 2025 alone, global data creation is projected to exceed 180 zettabytes you can see this in Figure 1.3, a figure so vast it defies intuitive comprehension. This data is not confined to tech giants or research labs—it flows through our daily lives: from smart thermostats adjusting room temperature to autonomous vehicles navigating city streets, and from personalized healthcare diagnostics to real-time financial trading platforms.

The implications are profound. Data is no longer just a byproduct of computation—it is the raw material of insight, innovation, and decision-making. Distributed systems make this possible by enabling scalable, fault-tolerant infrastructures that can store, process, and analyze data across geographies and domains. Understanding how data moves, transforms, and informs is essential for any computer scientist in the modern era.

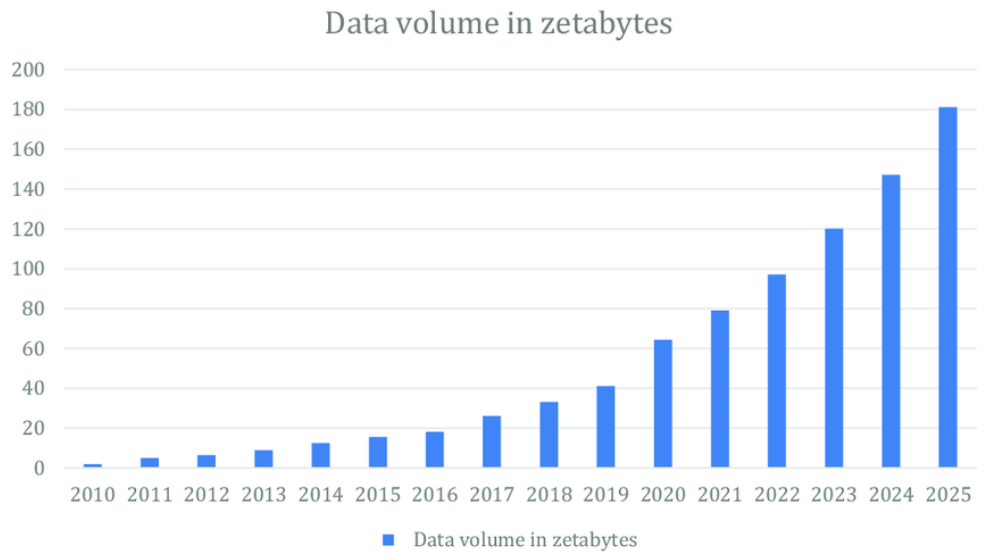


Figure 1.3: Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025

Yet, as the volume of data grows exponentially, so does the complexity of managing it. Storing and processing data at scale is one of the defining challenges of distributed systems. Traditional monolithic architectures crumble under the weight of petabytes of information, necessitating a shift to decentralized, fault-tolerant infrastructures. Distributed file systems like HDFS and object stores like Amazon S3 allow us to persist massive datasets reliably. Processing frameworks such as Apache Spark, Flink, and Hadoop enable parallel computation across clusters, turning raw data into actionable knowledge. Stream processing engines like Kafka and Pulsar handle real-time ingestion and transformation, ensuring that insights are not just accurate but timely.

This scale demands more than just technical prowess—it requires architectural foresight. Data must be partitioned intelligently, replicated for resilience, and processed with latency and throughput in mind. Systems must be elastic, adapting to fluctuating workloads, and secure, safeguarding sensitive information. As we move deeper into the era of machine learning and AI, the ability to store and process data at scale is not just a technical requirement—it is a strategic imperative.

1.1.3 Store and Process Data at Scale: Vertical vs Horizontal Scaling

As systems grow and user demands increase, scalability becomes a critical concern. Two primary strategies address this challenge: **vertical scaling** and **horizontal scaling**. Understanding their differences is essential for designing resilient, high-performance distributed systems.

Vertical scaling, also known as scaling up, involves enhancing the capacity of a single machine. This could mean upgrading the CPU, adding more RAM, or increasing disk throughput. It's straightforward and often requires minimal architectural changes. For example, a database server might be upgraded from 32 GB to 128 GB of RAM to handle more queries.
However, vertical scaling has limitations: hardware upgrades have diminishing returns, there's a physical ceiling to how much a single machine can be enhanced, and it introduces a single point of failure. If that machine crashes, the entire system goes down.

Horizontal scaling, or scaling out, takes a different approach. Instead of relying on a single powerful machine, it distributes the workload across multiple machines. These machines can be commodity servers, each handling a portion of the traffic or data. This strategy is foundational to distributed systems. It offers fault tolerance—if one node fails, others can take over—and elasticity, allowing systems to grow or shrink based on demand.
However, horizontal scaling introduces complexity: data must be partitioned, consistency must be managed, and communication between nodes becomes critical.

In practice, vertical scaling is often used for quick performance boosts or legacy systems, while horizontal scaling is preferred for modern cloud-native architectures. Distributed databases, microservices are all designed with horizontal scaling in mind. **You can see this video to let you visualize what we mean Horizontal vs Vertical Scaling?**

Advantages and Disadvantages

- **Vertical Scaling**
  - *Advantages*: Simple to implement, no need to redesign architecture.
  - *Disadvantages*: Limited scalability, single point of failure, expensive hardware.
- **Horizontal Scaling**
  - *Advantages*: High fault tolerance, cost-effective, scalable on demand.
  - *Disadvantages*: Complex architecture, requires load balancing and data distribution.

1.1.4 Historical Context

The roots of distributed computing trace back to the 1970s see Figure 1.4, when researchers began exploring how multiple computers could work together to solve problems more efficiently. Early systems like ARPANET laid the groundwork for networked communication, while the rise of UNIX and remote procedure calls (RPCs) enabled basic distributed interactions. Over time, the need for fault tolerance, scalability, and global reach led to the development of more sophisticated systems—culminating in today's cloud platforms, blockchain networks, and edge computing infrastructures.

1.1.5 Features of Distributed Systems

Distributed systems are distinguished by a set of core features that shape their architecture, behavior, and challenges. These features are not merely technical attributes—they define how distributed systems operate, scale, and recover in real-world environments. Understanding these characteristics is essential for designing systems that are robust, efficient, and responsive see Figure 1.5.

1. Geographical Separation

Distributed systems consist of components that are physically separated across different locations. These components may reside in different data centers, cities, or even continents. This separation

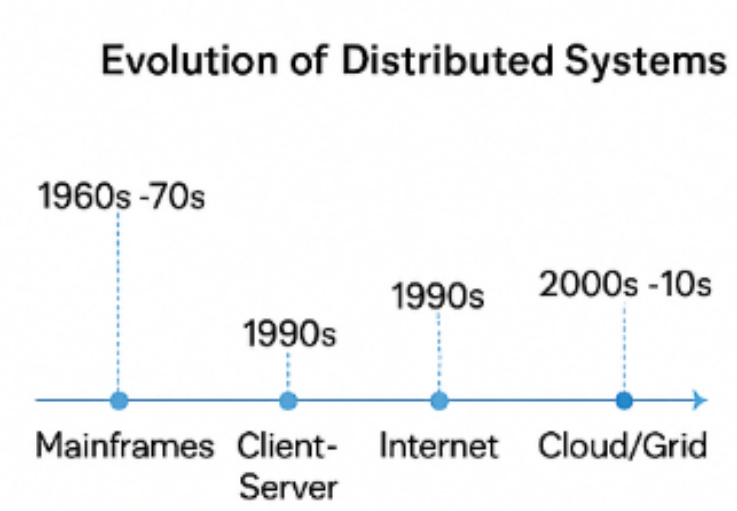


Figure 1.4: Evolution of Distributed Systems

enables global scalability and redundancy, but also introduces latency and network reliability concerns. For example, a content delivery network (CDN) caches data in multiple regions to serve users faster, but must synchronize updates across all nodes.

2. No Common Physical Clock

Unlike tightly coupled systems, distributed systems lack a shared global clock. Each node maintains its own local time, which can drift due to hardware differences or network delays. This absence of a common clock complicates event ordering and synchronization. To address this, distributed systems use logical clocks—such as Lamport timestamps or vector clocks—to infer causality and maintain consistency.

3. No Shared Physical Memory

Each node in a distributed system operates with its own local memory. There is no shared memory space accessible to all components, which means that communication must occur through message passing. This design promotes modularity and fault isolation, but requires careful coordination to ensure data consistency and coherence across nodes.

4. Autonomy and Heterogeneity

Nodes in a distributed system are autonomous—they can operate independently and may be managed by different organizations or teams. They are also heterogeneous, meaning they can run different hardware, operating systems, or software stacks. This diversity enhances flexibility and scalability, but also demands standardized communication protocols and robust interoperability mechanisms.

1.1.6 Why Distributed Systems?

The motivation for distributed systems stems from practical constraints and ambitious goals. Vertical scaling—adding more power to a single machine—faces diminishing returns due to hardware bottlenecks in memory, disk I/O, and processor throughput. Horizontal scaling, on the other hand,

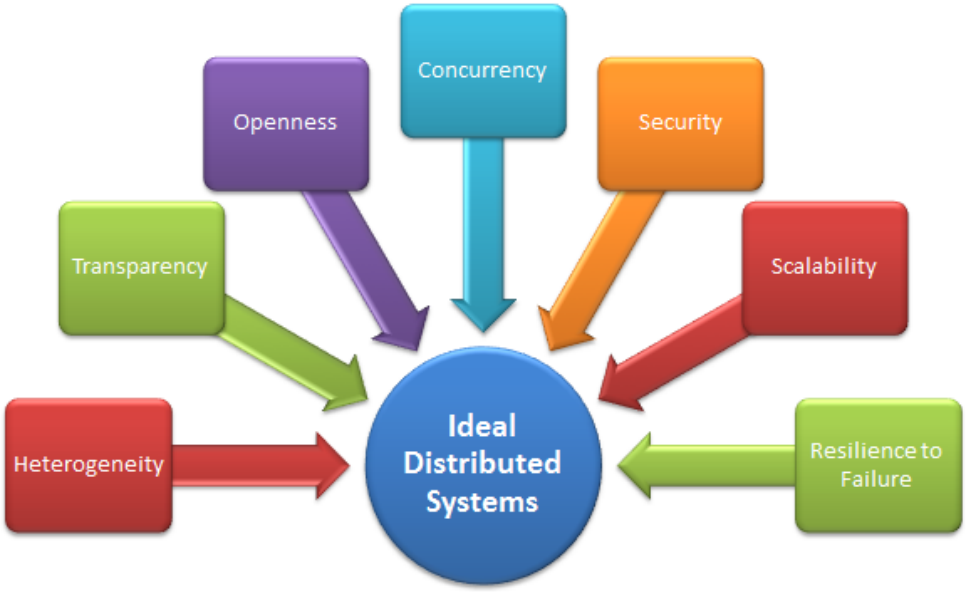


Figure 1.5: Illustration of Key Features in a Distributed System

distributes tasks across multiple machines, offering elasticity, fault isolation, and cost efficiency. But this shift introduces complexity: how do we synchronize processes, maintain consistency, and ensure availability when components can fail independently? why we don't use parallel systems and how they differ from distributed systems?

Parallel vs. Distributed Systems

Parallel and distributed systems both aim to improve computational efficiency, but they differ fundamentally in architecture and design philosophy. Parallel systems consist of multiple processors working simultaneously on a shared task, often within a single machine or tightly coupled environment. These systems rely on shared memory and typically operate under a unified clock, making synchronization straightforward but limiting scalability.

In contrast, distributed systems comprise independent nodes that communicate over a network. Each node has its own memory and clock, and coordination is achieved through message passing and consensus protocols. This design enables distributed systems to scale across geographic regions and tolerate faults more gracefully. While parallel systems excel in high-performance tasks like simulations and matrix computations, distributed systems are ideal for applications requiring resilience, scalability, and decentralized control—such as cloud platforms, microservices, and peer-to-peer networks see Figure 2.1 and Table 1.1 .

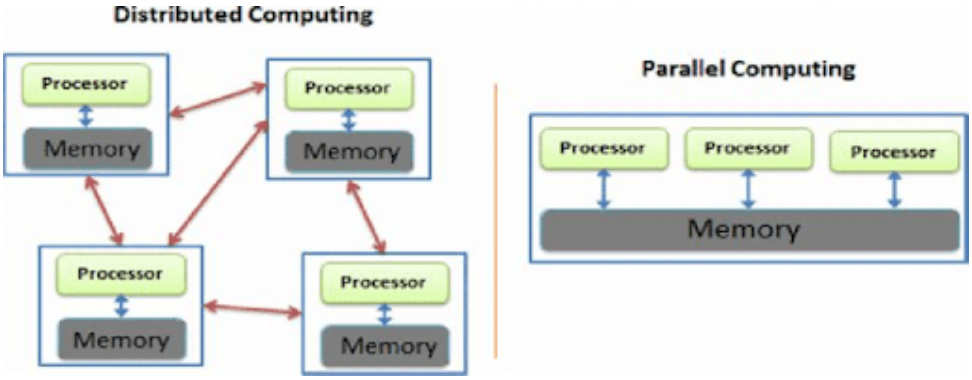


Figure 1.6: Parallel Vs. Distributed Systems

Table 1.1: Comparison Between Parallel and Distributed Systems

Aspect	**Parallel Systems**	**Distributed Systems**
Architecture	Tightly coupled processors sharing memory	Loosely coupled nodes with independent memory
Communication	Shared memory or high-speed interconnects	Message passing over a network
Clock Synchronization	Typically share a global clock	No global clock; use logical clocks
Location of Components	Usually located in the same physical machine or rack	Geographically dispersed across multiple locations
Fault Tolerance	Lower fault tolerance; failure may affect entire system	Higher fault tolerance; nodes can fail independently
Scalability	Limited scalability due to shared resources	Highly scalable across heterogeneous environments
Use Cases	Scientific computing, simulations, real-time processing	Web services, cloud computing, distributed databases
Resource Sharing	Shared memory and CPU resources	Independent resources coordinated via protocols

1.1.7 Motivating Case Study: Netflix – How Distributed Systems Make Streaming Work?

What Is Netflix Doing Behind the Scenes?

Netflix lets millions of people around the world watch movies and shows anytime they want. But how does it work so smoothly—even when millions are watching at the same time?

The answer: **distributed systems**. Netflix uses many computers working together across the globe to make sure videos load fast, recommendations are accurate, and the service doesn't crash.

Key Ideas in Netflix's System

Here are some simple examples of how Netflix uses distributed systems:

- **Content Delivery**: Netflix stores copies of videos in many places around the world. When you press play, it sends the video from a nearby server so it loads quickly.
- **Microservices**: Instead of one big program, Netflix is made of many small services. One handles your login, another gives you recommendations, another tracks what you've watched. These services work together like a team.
- **Handling Failures**: Netflix tests its system by pretending things go wrong—like a server crashing. This helps them make sure the system can recover and keep working.
- **Finding Services**: Netflix uses tools to help its services find each other, even when they move or change. This keeps everything connected.
- **Balancing Traffic**: If too many people use one server, Netflix spreads the load to others so no one gets stuck waiting.

Smart Recommendations

Netflix suggests shows you might like based on what you've watched. To do this, it collects data and runs smart algorithms across many computers. This helps it give you good suggestions quickly—even while millions of others are watching too.

Staying Strong Under Pressure

Netflix doesn't wait for problems to happen—they test their system by creating fake failures. This helps them build a system that can handle real problems without going down.

Why This Matters

Netflix is a great example of how distributed systems help real-world apps:

- They make things faster by using many computers in different places.
- They keep services running even when something breaks.
- They let companies grow and serve millions of users without slowing down.

1.2 Exercises

indexIntroduction!Sections

Q1. Define a distributed system in your own words. What makes it different from a centralized system?

Q2. Why is horizontal scaling generally preferred over vertical scaling in modern distributed architectures?

Q3. Describe one real-world application that relies on distributed systems. What challenges does it face?

Q4. Explain the role of message passing in distributed systems. Why is it necessary?

Q5. What are some historical milestones that contributed to the development of distributed computing?

1.3 Answers

indexIntroduction!Sections

A1. A distributed system is a network of independent computers that work together and appear as a single system to users. Unlike centralized systems, distributed systems rely on coordination and communication across multiple nodes.

A2. Horizontal scaling allows systems to grow by adding more machines, which is more cost-effective and fault-tolerant than upgrading a single machine. It supports elasticity and better handles large-scale workloads.

A3. Netflix is a prime example. It faces challenges like ensuring low-latency streaming, handling global traffic, and maintaining service availability during failures. Distributed caching and microservices help address these issues.

A4. Message passing enables nodes in a distributed system to communicate and coordinate actions. Since there's no shared memory or clock, messages are the only way to share state and synchronize behavior.

A5. Key milestones include the development of ARPANET, the introduction of RPCs, the rise of UNIX networking, and the emergence of cloud computing platforms like AWS and Azure.

1.4 Multiple-Choice Quiz

Q1. What is a key driver of the current data explosion?
- A. Decline in global internet usage
- B. Advances in gene sequencing and sensor technology
- C. Reduction in cloud storage prices
- D. Decrease in computing power

Q2. Which scaling method involves upgrading hardware on a single machine?
- A. Horizontal scaling
- B. Vertical scaling
- C. Distributed scaling
- D. Modular scaling

Q3. What is a major limitation of vertical scaling?
- A. It requires multiple machines
- B. It cannot improve cache performance
- C. It is constrained by physical hardware limits
- D. It eliminates the need for concurrency

Q4. Which cache level typically has the highest latency?
- A. L1
- B. L2
- C. L3
- D. Main memory

Q5. What is the approximate latency of accessing main memory compared to L1 cache?
- A. 5x slower
- B. 10x slower
- C. 100x slower
- D. 300x slower

Q6. Which of the following is true about Moore's Law in the context of processors?
- A. CPU and memory speeds grow at the same rate
- B. CPU speed has historically outpaced memory speed
- C. Memory speed grows faster than CPU speed
- D. Moore's Law applies only to memory

Q7. What is the main advantage of horizontal scaling?
- A. It reduces the need for distributed systems
- B. It allows data to be loaded faster using multiple machines
- C. It eliminates the need for concurrency models
- D. It simplifies memory management

Q8. Which of the following is NOT a requirement for building distributed systems?
- A. Naming protocols
- B. Synchronization mechanisms
- C. Shared physical memory
- D. Communication paradigms

Q9. What does virtualization help achieve in distributed systems?
- A. Reduced latency
- B. Increased heterogeneity
- C. Portability and flexibility
- D. Elimination of fault tolerance

Q10. What is a distributed system?
- A. A single computer with multiple cores
- B. A group of tightly coupled processors

C. A collection of independent computers that appear as one system
D. A cloud-based storage solution

Q11. **(Thinking)** Why is horizontal scaling generally preferred over vertical scaling in modern data-intensive applications?
A. It reduces the need for software updates
B. It allows scaling beyond hardware limits and improves fault tolerance
C. It simplifies cache design
D. It avoids concurrency issues

Q12. **(Thinking)** What challenge arises when scaling vertically with many cores on a chip?
A. Increased power consumption
B. Difficulty in delivering input data fast enough to all cores
C. Reduced memory latency
D. Elimination of interconnects

Q13. **(Thinking)** Why is fault tolerance a critical requirement in distributed systems?
A. Because distributed systems use unreliable hardware
B. Because partial failures are inevitable and must be handled gracefully
C. Because all nodes share the same memory
D. Because it improves encryption

Q14. **(Thinking)** How does the processor-memory speed gap affect system design?
A. It encourages the use of slower CPUs
B. It necessitates better caching and parallelism strategies
C. It eliminates the need for distributed systems
D. It simplifies vertical scaling

Q15. **(Thinking)** What architectural implication arises from the need to load 4TB of data in minutes?
A. Use of larger disks
B. Use of more CPU cores
C. Use of distributed machines with parallel I/O channels
D. Use of deeper cache hierarchies

Q16. Which feature is common to distributed systems but not to parallel systems?
A. Shared physical memory
B. Common physical clock
C. Geographical separation
D. Homogeneity

Q17. Which of the following is a communication paradigm in distributed systems?
A. Shared memory
B. Message passing
C. Direct disk access
D. Cache coherence

2. Communication Paradigms

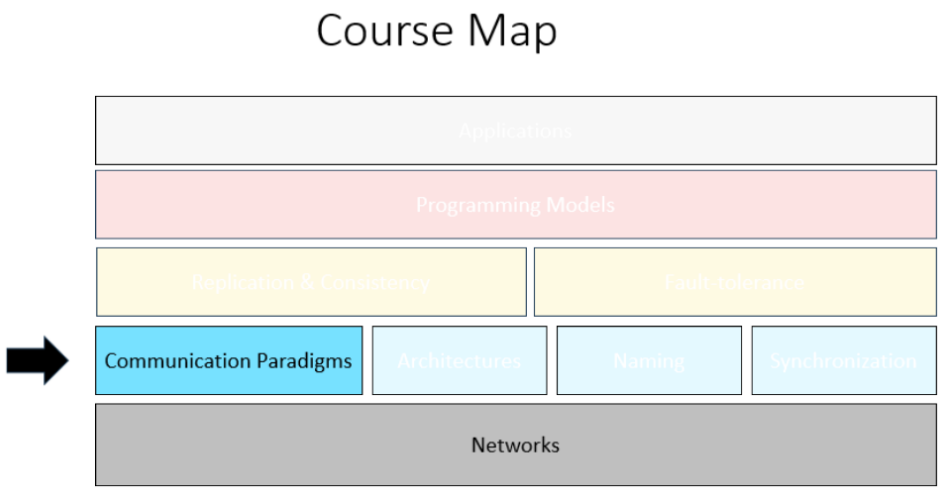


Figure 2.1: Communication Paradigms

2.1 Introduction to Remote Invocation in Distributed Systems

Distributed systems consist of multiple independent computers that collaborate to perform tasks. Communication between these entities is essential, and remote invocation is a powerful abstraction that allows a process to execute code on another machine as if it were local.

This chapter explores the concept of remote invocation, its role in distributed systems, and the foundational ideas behind Remote Procedure Calls (RPCs). We begin by examining the types of entities involved and the communication paradigms that enable their interaction.

2.2 Communicating Entities

Entities in distributed systems can be broadly classified into:

- **System-oriented entities:** These include processes, threads, and nodes. They represent the underlying execution units and hardware components.
- **Problem-oriented entities:** These are higher-level abstractions such as objects in object-

oriented programming. They encapsulate behavior and state relevant to the application domain.

Understanding how these entities communicate is crucial for designing robust distributed applications.

2.3 Communication Paradigms

Communication paradigms define the methods by which entities exchange information. They can be categorized based on the location of the communicating entities:

1. **Same Address Space:** Entities share memory and can communicate via global variables or direct procedure calls.
2. **Same Computer, Different Address Spaces:** Communication occurs through files, signals, or shared memory.
3. **Networked Computers:** Entities communicate over a network using sockets or remote invocation mechanisms.

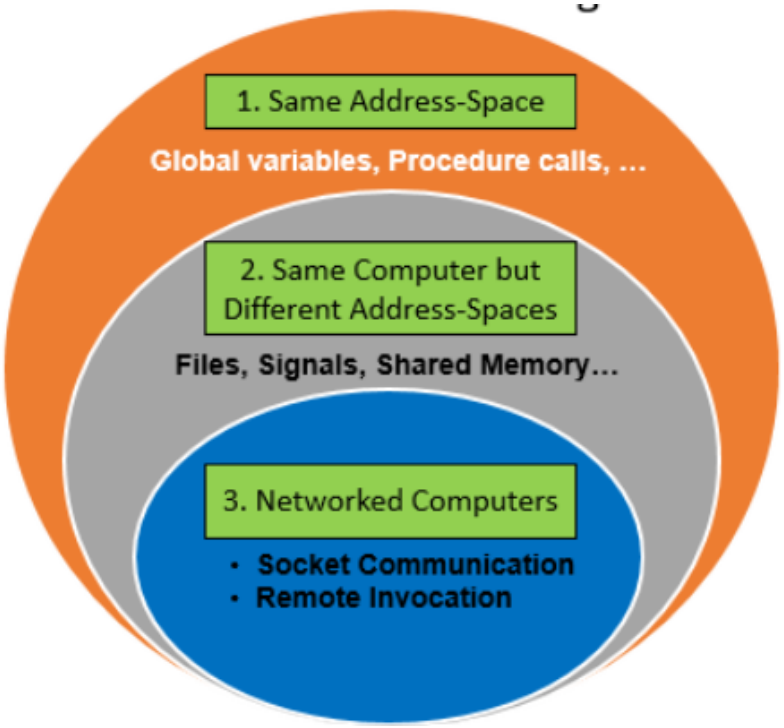


Figure 2.2: Communication paradigms based on entity location

This chapter focuses on the third category—communication between entities on networked computers.

2.4 Middleware Layers in Distributed Systems

Middleware is a software layer that resides between the operating system and distributed applications. It abstracts the complexity of network communication, heterogeneity, and resource management, enabling seamless interaction among distributed components.

Overview of Middleware Architecture

The middleware stack typically consists of several layers, each responsible for a specific aspect of distributed communication and coordination:

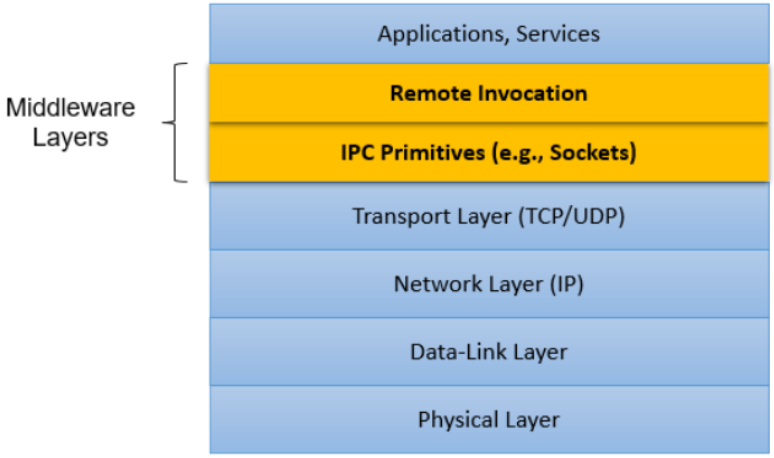


Figure 2.3: Layered middleware architecture in distributed systems

Layer Descriptions

1. **Application Layer**
   - Contains user-facing applications and services.
   - Relies on middleware to access remote resources and services.
2. **Remote Invocation Layer**
   - Provides abstractions such as Remote Procedure Call (RPC) and Remote Method Invocation (RMI).
   - Handles marshalling, unmarshalling, and stub generation.
3. **Communication Services Layer**
   - Offers messaging, queuing, and event-driven communication.
   - Examples include message brokers, publish-subscribe systems, and service buses.
4. **Resource Management Layer**
   - Manages distributed resources such as databases, filesystems, and compute nodes.
   - Supports load balancing, replication, and fault tolerance.
5. **Security and Policy Layer**
   - Enforces authentication, authorization, and encryption.
   - Manages access control policies and audit trails.
6. **Transport Layer**
   - Provides reliable or best-effort data transmission.
   - Protocols include TCP, UDP, and QUIC.
7. **Network Layer**
   - Handles routing and addressing across distributed nodes.
   - Protocols include IP, ICMP, and routing algorithms.
8. **Data-Link and Physical Layers**
   - Responsible for physical transmission of data over hardware.
   - Includes Ethernet, Wi-Fi, and optical links.

Benefits of Layered Middleware

The layered architecture of middleware in distributed systems offers a structured approach to managing complexity. By separating concerns across distinct layers, developers can build scalable, maintainable, and interoperable systems. Each layer encapsulates specific functionality—ranging from transport protocols to application-level abstractions—allowing independent development and

optimization. This modularity not only simplifies system design but also enhances flexibility when integrating diverse technologies and platforms.

- **Modularity:** Each layer can evolve independently, enabling focused improvements and easier debugging.
- **Interoperability:** Middleware supports communication between heterogeneous systems, programming languages, and operating environments.
- **Scalability:** Layered design facilitates dynamic resource allocation, load balancing, and horizontal scaling.
- **Maintainability:** Clear separation of concerns simplifies system upgrades, testing, and long-term maintenance.
- **Modularity:** Each layer can evolve independently.
- **Interoperability:** Supports heterogeneous platforms and languages.
- **Scalability:** Facilitates dynamic resource allocation and load balancing.
- **Maintainability:** Simplifies debugging and system upgrades.

Examples of Middleware Technologies

- **CORBA:** Common Object Request Broker Architecture
- **Java RMI:** Remote Method Invocation in Java
- **gRPC:** Google's high-performance RPC framework
- **Message Brokers:** RabbitMQ, Apache Kafka
- **Enterprise Middleware:** .NET Remoting, EJB, WebSphere

2.5 Introduction to Remote Invocation Concept

Remote invocation is a fundamental abstraction in distributed systems that allows a process to execute a procedure or method on a remote machine as if it were local. This concept simplifies distributed programming by hiding the complexities of network communication, data serialization, and transport protocols.

In traditional local procedure calls, both caller and callee reside in the same address space. However, in distributed systems, the caller and callee may be located on different machines, possibly running different operating systems and architectures. Remote invocation bridges this gap by providing a seamless interface for invoking remote functionality.

The key idea behind remote invocation is that the programmer does not need to explicitly manage sockets, message formats, or network protocols. Instead, the middleware handles these details transparently. This abstraction enables developers to focus on application logic while relying on the system to manage communication.

Remote invocation is typically implemented in two forms:

- **Remote Procedure Call (RPC):** Allows invoking procedures on remote systems using a procedural interface.
- **Remote Method Invocation (RMI):** Extends RPC to object-oriented environments, enabling method calls on remote objects.

Advantages of Remote Invocation

- **Transparency:** Developers write code as if invoking local procedures.
- **Modularity:** Promotes separation of concerns and reuse of components.
- **Interoperability:** Supports communication across heterogeneous platforms.
- **Scalability:** Enables distributed deployment and load balancing.

Remote invocation is a cornerstone of modern distributed systems, forming the basis for service-oriented architectures, microservices, and cloud-native applications.

Example: Imagine a weather app that fetches temperature data from a remote server. Instead of manually opening a socket and formatting a request, the app simply calls `getTemperature("Cairo")`—and the middleware handles the rest.

2.5.1 Remote Procedure Call (RPC)

RPC allows a client to invoke a procedure on a remote server using a familiar function-call syntax.

Example: A client wants to add two numbers remotely:

```
int result = add(5, 3); // Looks local, but runs on server
```

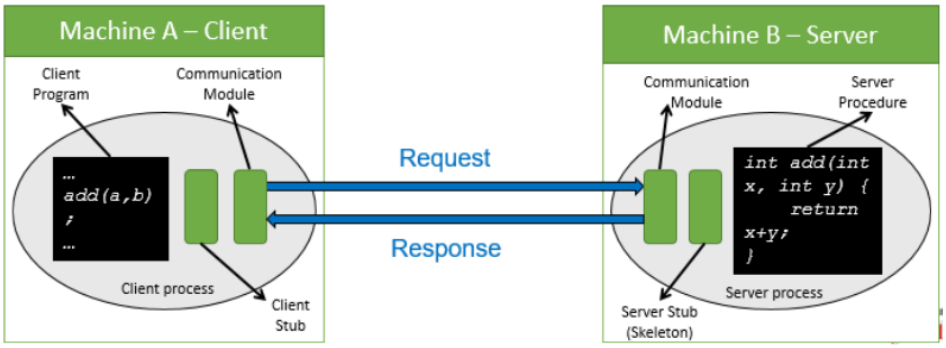


Figure 2.4: RPC add two numbers Example

RPC Architecture

- **Client Stub:** Marshals parameters and sends request.
- **Server Stub:** Unmarshals request and invokes procedure.
- **Transport Module:** Handles message delivery.

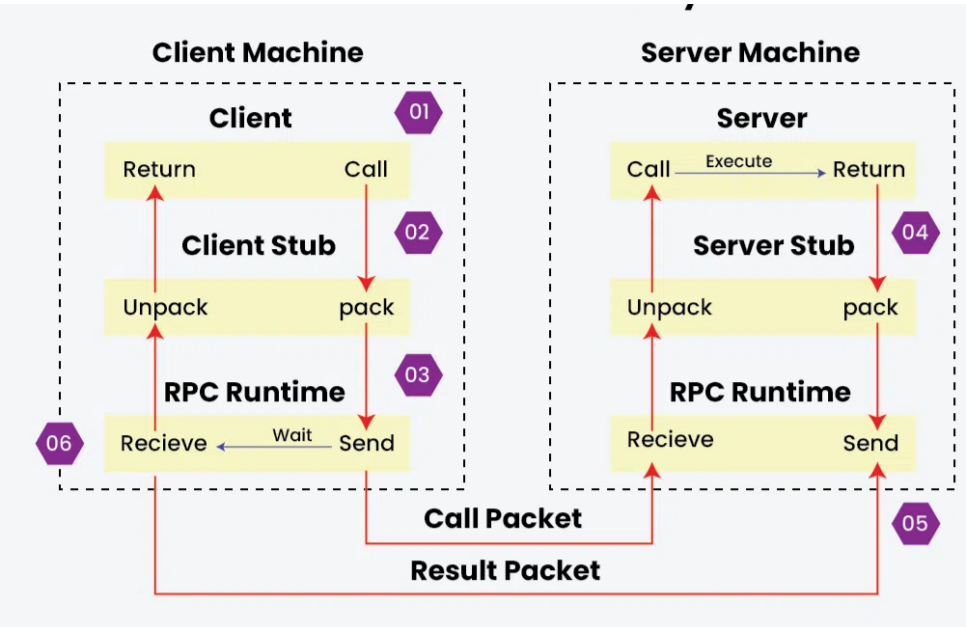


Figure 2.5: RPC architecture showing client and server interaction

Client Stub Workflow in RPC

In a Remote Procedure Call (RPC) system, the client stub plays a pivotal role in abstracting the complexity of remote communication. When a client initiates a procedure call, it does so as if the procedure were local. The client stub intercepts this call and begins by *marshalling* the procedure name, arguments, and metadata into a structured call packet. This packet is then handed off to the client-side RPC runtime, which transmits it across the network to the server. Upon receiving a response from the server—typically in the form of a result packet—the client-side RPC runtime delivers it back to the client stub. The stub then *unmarshals* the result, converting it into a format suitable for the client application, and returns it as though the procedure had executed locally. This seamless workflow allows distributed systems to maintain transparency and modularity, enabling developers to invoke remote services without needing to manage low-level networking details.

Server Stub Workflow in RPC

On the server side of an RPC system, the server stub is responsible for receiving and processing incoming procedure calls from remote clients. When the server-side RPC runtime receives a call packet, it forwards the request to the appropriate server stub. The stub begins by *unmarshalling* the packet, extracting the procedure name and arguments. It then invokes the corresponding procedure implementation on the server, passing in the extracted arguments. Once the procedure completes execution, the server stub *marshals* the result into a response packet. This packet is then sent back through the server-side RPC runtime to the client. The server stub thus acts as a bridge between the network layer and the actual server logic, ensuring that remote calls are correctly interpreted and executed within the server's address space.

Challenges in RPC Systems

Remote Procedure Call (RPC) simplifies distributed programming by allowing procedures to be invoked across machines as if they were local. However, this abstraction introduces several challenges that are critical to understand, especially for beginners. These challenges include parameter marshaling, data representation, and failure independence.

1. Parameter Passing via Marshaling Marshaling is the process of converting procedure parameters and return values into a format suitable for transmission over the network.

For example, when a client calls `add(3, 5)`, the integers `3` and `5` must be packed into a message, sent to the server, and then unpacked for execution. This becomes complex when dealing with reference parameters, which are valid only in the client's memory space.

To preserve semantics, RPC systems use a *copy-restore* strategy: the referenced data is copied to the server, and any changes are copied back to the client after execution.

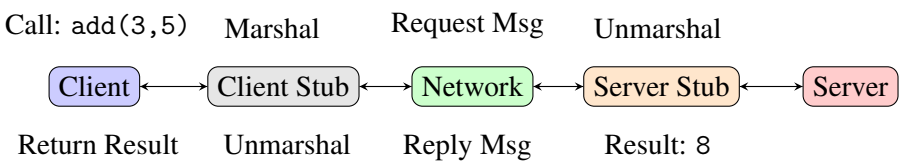


Figure 2.6: RPC marshaling and unmarshaling workflow

Another Simple Example: Sending an integer over the network:

```
int x = 42;
char* msg = serialize(x); // Marshalling
send(msg);
```

2. Data Representation Across Architectures Clients and servers may run on different architectures. For example, Intel uses little-endian format, while SPARC uses big-endian. RPC

systems must standardize data formats to ensure correct interpretation.

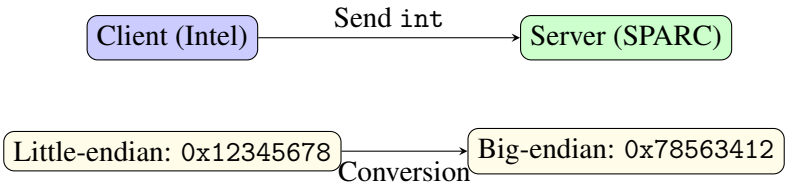


Figure 2.7: Data representation differences across architectures

3. Failure Independence In RPC, the client and server are separate entities. If the server crashes or the network fails, the client must handle these failures gracefully.

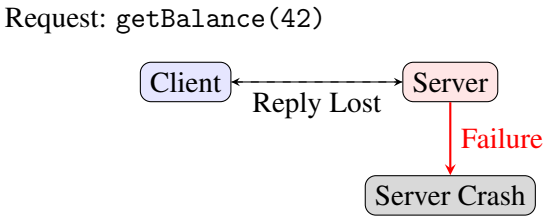


Figure 2.8: Failure independence in RPC systems

Example Consider a banking application where a client invokes `getBalance(accountID)`. If the server crashes after processing the request but before sending the reply, the client may retry the call. This could accidentally repeat the operation—such as charging a customer twice—unless the system is designed to safely handle repeated requests. To avoid such problems, RPC systems often use timeouts, retries, and unique request identifiers to ensure safe and consistent behavior.

These challenges highlight the importance of designing RPC systems with careful attention to serialization formats, error handling, and communication protocols. For beginners, understanding these pitfalls is essential to building reliable distributed applications.

Tip: Always use standardized formats like XDR or Protocol Buffers to avoid compatibility issues.

Remote Procedure Call Types

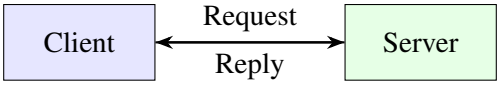
Remote Procedure Calls (RPCs) allow a client to invoke a procedure on a remote server. Depending on how the client and server interact, RPCs can be classified into three main types.

1. Synchronous RPC

In synchronous RPC, the client sends a request and waits (blocks) until the server processes the request and returns the result.

Explanation This is the most common type of RPC. It behaves like a regular function call: the client pauses until the server responds. While simple, it can be inefficient if the server takes time to respond.

Example A banking app sends a request to check account balance. The client waits until the server replies with the balance.



Client behavior: Blocks until reply is received.

2. Asynchronous RPC

In asynchronous RPC, the client sends a request and receives an immediate acknowledgment (ACK). The client does not wait for the result and continues execution.

Explanation This is useful when the client does not need the result immediately or at all. It improves responsiveness and resource usage.

Example A sensor device sends temperature data to a server. It doesn't wait for confirmation—it just sends and moves on.

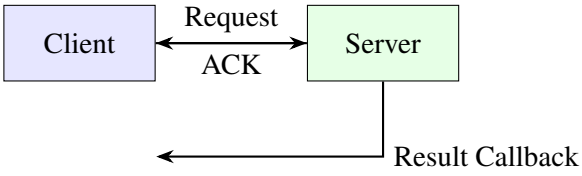
Client behavior: Continues execution after ACK.

3. Deferred Synchronous RPC

In deferred synchronous RPC, the client sends a request and receives an ACK. Later, the server sends the result back using a second RPC call.

Explanation This is useful when the client wants the result but doesn't want to block. It splits the interaction into two phases: request and callback.

Example A user submits a video for processing. The server acknowledges the request immediately. Once processing is done, the server notifies the client with the result.



Client behavior: Continues execution and receives result later.

Mini-Project: Simple Chat Using RPC

Objective Create a basic RPC system where a client sends a chat message and the server replies with a confirmation.

Setup Install the required Python packages:

Install Dependencies

```
pip install grpcio grpcio-tools
```

Step 1: Define the Chat Service

Create a file named `chat.proto`:

chat.proto

```
syntax = "proto3";

service ChatService {
  rpc SendMessage (ChatRequest) returns (ChatReply);
}

message ChatRequest {
  string user = 1;
  string message = 2;
}

message ChatReply {
  string response = 1;
}
```

Compile the proto file:

Compile Proto

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. chat.
    proto
```

Step 2: Server Code

Create a file named `server.py`:

server.py

```
import grpc
from concurrent import futures
import chat_pb2
import chat_pb2_grpc

class ChatServer(chat_pb2_grpc.ChatServiceServicer):
    def SendMessage(self, request, context):
        print(f"{request.user} says: {request.message}")
        return chat_pb2.ChatReply(response=f"Message received from {
            request.user}")

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=2))
    chat_pb2_grpc.add_ChatServiceServicer_to_server(ChatServer(), server
        )
    server.add_insecure_port('[::]:50051')
    server.start()
    print("Chat server running...")
    server.wait_for_termination()

if __name__ == '__main__':
    serve()
```

Step 3: Client Code

Create a file named `client.py`:

client.py

```
import grpc
import chat_pb2
import chat_pb2_grpc

def run():
    channel = grpc.insecure_channel('localhost:50051')
    stub = chat_pb2_grpc.ChatServiceStub(channel)

    response = stub.SendMessage(chat_pb2.ChatRequest(user="Alice",
        message="Hello!"))
    print(response.response)

if __name__ == '__main__':
    run()
```

Running the Project

1. Compile the `chat.proto` file.
2. Run the server:

Run Server

```
python server.py
```

3. Run the client:

Run Client

```
python client.py
```

Expected output:

Sample Output

```
Message received from Alice
```

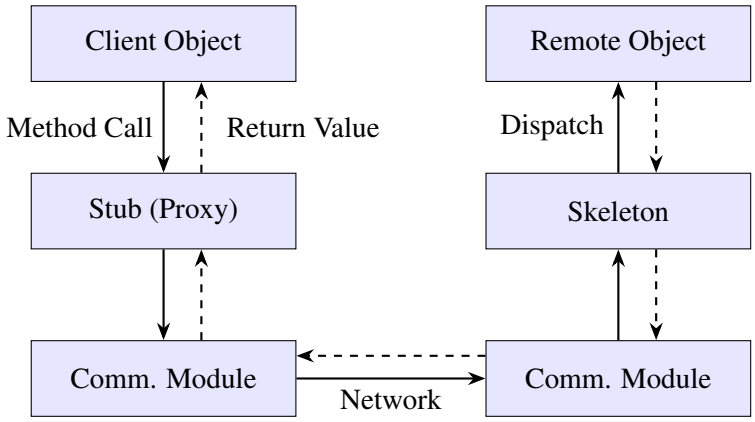
2.6 Remote Method Invocation

Remote Method Invocation (RMI) extends Remote Procedure Call (RPC) into the object-oriented domain. It allows objects on different machines to invoke each other's methods as if they were local, supporting distributed object-based systems. Remote Method Invocation (RMI) is a Java API that allows an object running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. This is useful for building distributed applications.

Key Concepts

Concept	Description
Remote Object	An object whose methods can be invoked remotely.
Remote Interface	Interface defining remotely accessible methods.
Proxy (Stub)	Local representative of the remote object.
Skeleton & Dispatcher	Server-side components that handle incoming method calls.
Remote Reference Module	Translates between local and remote object references.

RMI Control Flow Diagram



Step-by-Step Example: Remote Calculator

1. Remote Interface

Calculator.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

2. Remote Implementation

CalculatorImpl.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class CalculatorImpl extends UnicastRemoteObject
    implements Calculator {
    public CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) {
        return a + b;
    }
}
```

3. Server Setup

CalculatorServer.java

```
import java.rmi.Naming;

public class CalculatorServer {
    public static void main(String[] args) throws Exception {
        Calculator calc = new CalculatorImpl();
        Naming.rebind("CalculatorService", calc);
        System.out.println("CalculatorService is running...");
    }
}
```

4. Client Code

CalculatorClient.java

```
import java.rmi.Naming;

public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        Calculator calc = (Calculator) Naming.lookup("rmi://
            localhost/CalculatorService");
        System.out.println("Result: " + calc.add(5, 3));
    }
}
```

2.7 Exercises

Section A: Conceptual Understanding

1. Define Remote Procedure Call (RPC). What distinguishes RPC from traditional local procedure calls?
2. Explain the role of the client stub in RPC. What are its responsibilities during a remote invocation?
3. List and describe the challenges in implementing RPC. Include at least three distinct challenges.
4. Differentiate between synchronous and asynchronous RPCs. How does each affect client-side execution?
5. What is parameter marshaling? Why is it necessary in distributed systems?

Section B: Application & Analysis

6. **Scenario:** A client sends a reference parameter to a server via RPC.
   - What issues might arise?
   - How can they be resolved?
7. **Diagram Interpretation:** Based on the RPC flow diagram, describe the sequence of actions from client call to server response.
8. Why is failure handling more complex in RPC than in local procedure calls?
9. Explain how differences in architecture (e.g., little-endian vs. big-endian) affect RPC communication.
10. Compare RPC and RMI. What additional capabilities does RMI provide over RPC?

2.8 Answers

Section A: Conceptual Understanding

1. **RPC Definition:** RPC allows a program to execute a procedure on a remote machine as if it were local, abstracting away the communication details.
2. **Client Stub Role:**
   - Acts as a proxy for the remote procedure.
   - Marshals parameters into a request message.
   - Sends the request via transport routines.
   - Unmarshals the reply and returns results to the caller.
3. **RPC Challenges:**
   - **Parameter marshaling:** Converting parameters into transmittable formats.
   - **Data representation:** Ensuring uniform formats across heterogeneous systems.
   - **Failure independence:** Handling crashes and network failures gracefully.
4. **Synchronous vs. Asynchronous RPCs:**
   - **Synchronous:** Client blocks until server responds.
   - **Asynchronous:** Client continues execution after sending request; server may respond later.
5. **Parameter Marshaling:** The process of packing parameters into a message for transmission. It ensures compatibility and correctness across different systems.

Section B: Application & Analysis

6. **Reference Parameter Issues:**
   - **Invalid references:** Server cannot access client memory.
   - **No reflection of changes:** Updates on server don't affect client.
   - **Solution:** Use copy/restore semantics — copy data to server, and copy back results.
7. **RPC Flow Sequence:**
   - Client calls stub $\rightarrow$ stub marshals data $\rightarrow$ sends request $\rightarrow$ server stub receives $\rightarrow$ unmarshals $\rightarrow$ executes procedure $\rightarrow$ marshals result $\rightarrow$ sends response $\rightarrow$ client stub receives $\rightarrow$ returns result.
8. **Failure Independence Complexity:** Remote systems can fail independently. Clients must handle server crashes, network issues, and partial failures — unlike local calls where both components share fate.
9. **Architecture Differences:** Data types may vary in size and format. For example, integers may be 4 bytes on one system and 8 on another. Endianness affects byte order, requiring standardization in message formats.
10. **RPC vs. RMI:**
    - **RPC:** Works with procedures and basic data types.
    - **RMI:** Supports object references and method calls in object-oriented systems, enabling richer interactions.

2.9 Multiple-Choice Quiz

Q1. What is the primary purpose of Remote Procedure Calls (RPC)?
- A. To execute procedures locally
- B. To abstract remote communication as local procedure calls
- C. To manage memory across distributed systems
- D. To provide direct access to remote hardware

Q2. Which of the following is NOT a challenge in implementing RPC?
- A. Parameter marshaling
- B. Data representation differences
- C. Failure independence
- D. Direct memory access between client and server

Q3. What is the role of the client stub in RPC?
- A. Execute server-side logic
- B. Marshal parameters and send request
- C. Handle server failures
- D. Manage server memory

Q4. What does marshaling refer to in RPC?
- A. Encrypting parameters
- B. Packing parameters for transmission
- C. Executing remote code
- D. Managing memory allocation

Q5. What distinguishes synchronous from asynchronous RPC?
- A. Synchronous RPC blocks the client until response
- B. Asynchronous RPC uses encryption
- C. Synchronous RPC avoids marshaling
- D. Asynchronous RPC is only used locally

Q6. What is the 'Copy/Restore' method used for?
- A. Encrypting reference parameters
- B. Passing reference parameters safely
- C. Avoiding marshaling
- D. Improving performance

Q7. What is the role of the server stub?
- A. Execute client-side logic
- B. Unmarshal parameters and invoke procedure
- C. Handle client failures
- D. Manage network connections

Q8. Why is data representation a challenge in RPC?
- A. All systems use identical formats
- B. Different architectures represent data differently
- C. RPC avoids data transmission
- D. Data representation is irrelevant in RPC

Q9. What does the Remote Reference Module do in RMI?
- A. Encrypts remote calls
- B. Translates object references across machines
- C. Executes remote procedures
- D. Handles marshaling

Q10. What advantage does RMI have over traditional RPC?
- A. Faster execution
- B. Supports object references
- C. Avoids marshaling

D. Requires no stubs

Q11. Which scenario best illustrates synchronous RPC limitations?

A. Client receives immediate response
B. Client blocks, causing UI freeze
C. Server handles multiple requests
D. Client uses callback

Q12. Why choose asynchronous RPC in microservices?

A. Simplifies debugging
B. Reduces memory usage
C. Improves scalability and responsiveness
D. Avoids marshaling

Q13. What trade-off does 'Copy/Restore' introduce?

A. Performance vs correctness
B. Simplicity vs abstraction
C. Correctness vs overhead
D. Speed vs encryption

Q14. When does marshaling fail to preserve semantics?

A. Passing primitive types
B. Passing shared memory pointers
C. Passing strings
D. Passing binary blobs

Q15. What abstraction benefit of RPC can mislead developers?

A. Exposes network details
B. Hides latency and failure risks
C. Requires manual memory management
D. Forces synchronous calls

Q16. What happens if the server crashes during an RPC?

A. Client receives partial response
B. Client blocks indefinitely unless timeout is handled
C. Server automatically retries
D. Client switches to local execution

Q17. Why is idempotency important in RPC design?

A. It ensures encryption
B. It allows safe retries without side effects
C. It improves marshaling
D. It avoids blocking

Q18. Which design choice improves fault tolerance in RPC systems?

A. Using synchronous calls only
B. Avoiding retries
C. Implementing timeouts and retries
D. Disabling marshaling

Q19. What is a drawback of hiding remote communication behind local call abstraction?

A. Developers overestimate latency
B. Developers ignore network failures
C. Developers avoid using RPC
D. Developers must manage memory manually

Q20. Which of the following best describes the role of interface definition in RPC?

A. It defines encryption protocols
B. It specifies procedure signatures for stub generation
C. It manages network connections

D. It handles marshaling logic

Q21. Which of the following best explains why RPC abstraction can lead to performance issues in distributed systems?

A. It forces developers to use synchronous calls only.
B. It hides network latency and failure risks, encouraging misuse of remote calls.
C. It requires manual memory management across machines.
D. It prevents the use of asynchronous communication.

Q22. A developer notices that retrying a failed RPC causes duplicate entries in a database. What is the most appropriate solution?

A. Use asynchronous RPC to avoid blocking.
B. Encrypt the RPC payload to prevent duplication.
C. Make the RPC operation idempotent.
D. Increase the server's timeout threshold.

Q23. Why is passing pointers between client and server problematic in RPC?

A. Pointers are encrypted during transmission.
B. Pointers reference memory addresses that are invalid across address spaces.
C. Pointers are automatically marshaled by the stub.
D. Pointers are converted to strings before transmission.

Q24. In designing an RPC system for real-time applications, which factor is most critical?

A. Minimizing interface definition complexity.
B. Guaranteeing low-latency and predictable response times.
C. Using synchronous calls for consistency.
D. Avoiding marshaling to reduce overhead.

3. Architectures Models in Distributed Systems

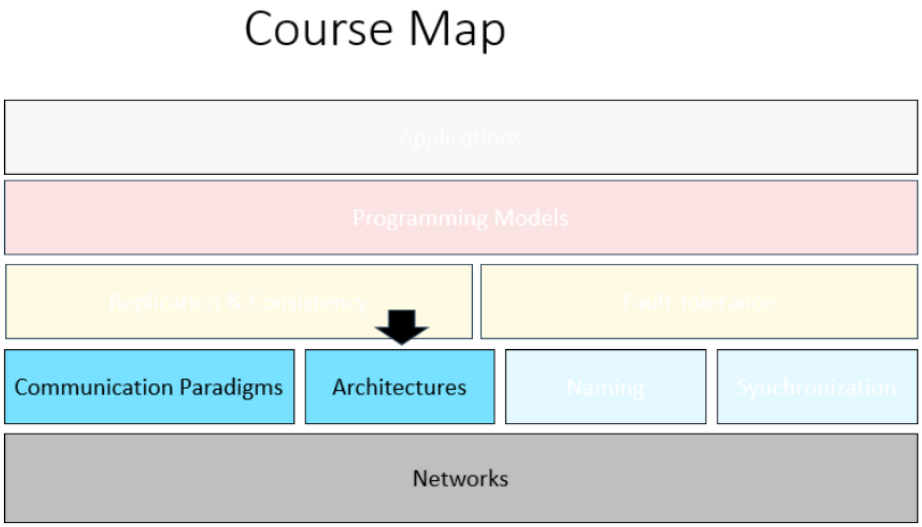


Figure 3.1: Architectures

Distributed systems consist of multiple independent computers that work together to perform tasks. The way these computers are organized—how they communicate, coordinate, and share responsibilities—is defined by **the system's architecture**.

Understanding architectural models is essential for designing systems that are scalable, fault-tolerant, and efficient. This chapter introduces key architectural paradigms, including Master-Slave, Peer-to-Peer, and Hybrid models, along with structural patterns like tiering and layering.

Analogy: Booking a flight online involves multiple systems—user interface, search engine, databases—all working together. Their organization is the architecture.

Characterizing Distributed Systems

Distributed systems can be described along three dimensions:

- **Entities**: Clients, servers, peers—who participates?
- **Communication Paradigms**: Sockets, RPC—how do they talk?

- **Roles and Responsibilities**: Coordination, computation, storage—what do they do?

Analogy: A team project with leaders, researchers, writers, and editors. The way tasks are assigned and coordinated is the architecture.

3.1 Architectural Models

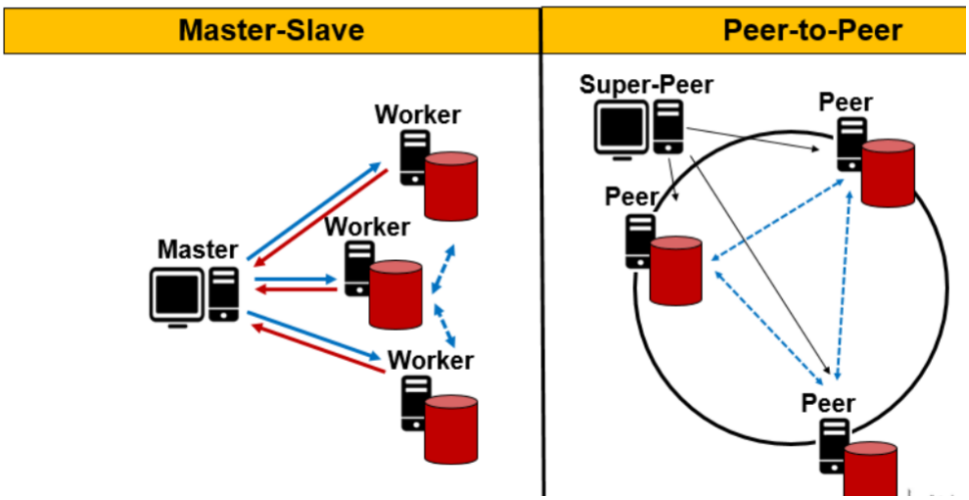


Figure 3.2: Architectural Models

3.1.1 Master-Slave Architecture

The Master-Slave architecture is one of the most traditional and straightforward organizational models in distributed systems. In this setup, a single node—referred to as the *master*—acts as the central coordinator, while the remaining nodes—called *slaves* or *workers*—carry out tasks assigned by the master. This hierarchical structure simplifies decision-making, as all control and scheduling responsibilities are centralized.

Characteristics:

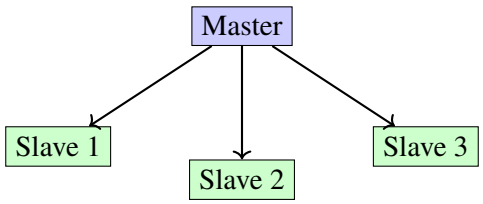
- Hierarchical structure
- Centralized decision-making
- Vulnerable to single point of failure
- Limited scalability

Example 1: A teacher (master) assigns homework to students (slaves), who complete the tasks and report back.

Example 2: A web server handling multiple client requests. If the server fails, the system halts.

Trend: Still used in systems like database replication and job scheduling, but often replaced by more fault-tolerant models.

Diagram:



3.1.2 Peer-to-Peer Architecture

In contrast to the hierarchical Master-Slave model, Peer-to-Peer (P2P) architecture distributes responsibilities equally among all participating nodes. Each node, or *peer*, acts both as a client and a server, capable of initiating requests and responding to others. This decentralized structure enhances fault tolerance and scalability.

Characteristics:

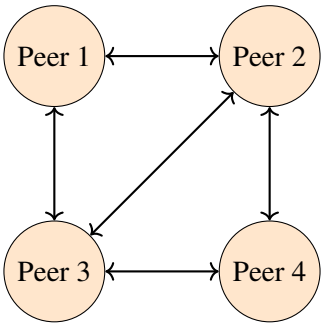
- Decentralized structure
- High fault tolerance
- Excellent scalability

Example 1: A group of friends sharing files directly with each other without a central server.

Example 2: BitTorrent—users download and share file pieces with each other.

Trend: Widely used in blockchain networks, decentralized storage, and collaborative platforms.

Diagram:



3.1.3 Hybrid Architecture

Hybrid architecture combines elements of both Master-Slave and Peer-to-Peer models to leverage the strengths of each. Typically, it features a set of master nodes that manage coordination and metadata, while peer nodes handle actual data exchange and computation.

Characteristics:

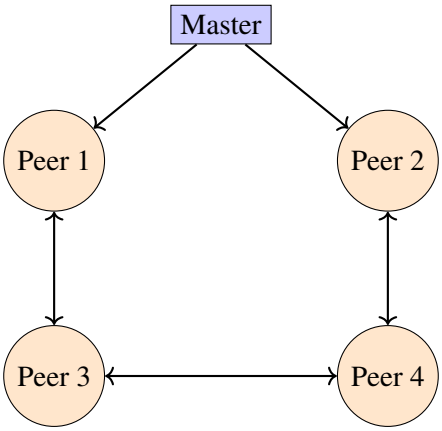
- Semi-centralized coordination
- Balanced fault tolerance
- Good scalability

Example 1: A school system where administrators (masters) manage records, while students (peers) collaborate on projects.

Example 2: Skype—central server helps connect users, but calls are peer-to-peer.

Trend: Common in cloud systems, distributed file systems (e.g., HDFS), and large-scale web applications.

Diagram:



3.2 Types of Peer-to-Peer Architectures

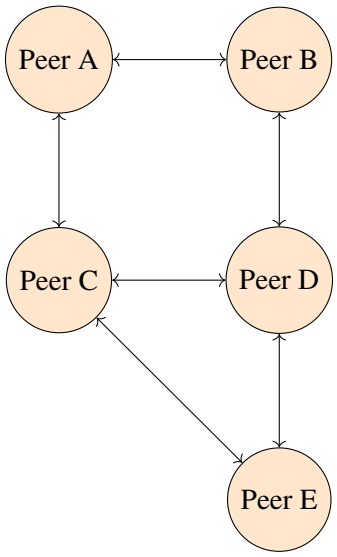
Peer-to-Peer (P2P) architecture represents a paradigm shift from traditional client-server models by decentralizing control and distributing responsibilities across all participating nodes. In a P2P system, each node—called a *peer*—can act both as a client and a server, initiating requests and responding to others. This design promotes scalability, fault tolerance, and resource sharing, making it ideal for applications like file sharing, blockchain, collaborative platforms, and decentralized storage. Unlike centralized systems, where a single server may become a bottleneck or point of failure, P2P networks thrive on dynamic participation and resilience. However, not all P2P systems are built the same. Depending on how peers are organized and how data is located, P2P architectures can be classified into several types, each with its own trade-offs in efficiency, complexity, and robustness.

3.2.1 Unstructured P2P Architecture

Unstructured P2P systems are the most flexible and decentralized form of peer-to-peer architecture. In this model, peers connect randomly without any global organization or indexing. When a peer wants to find data, it typically floods the network with queries, asking neighboring nodes to forward the request until the data is found. This approach is simple to implement and highly resilient to peer churn (nodes joining or leaving), but it can be inefficient, especially for rare data.

Example: Gnutella file-sharing network. **Analogy:** Asking everyone in a crowded room if they've seen your lost keys. **Trend:** Less common today due to inefficiency, but useful in highly dynamic environments.

Diagram:

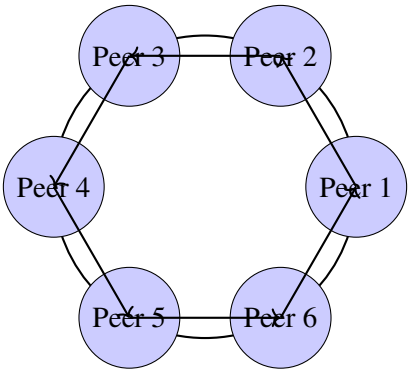


3.2.2 Structured P2P Architecture

Structured P2P systems introduce organization by using distributed hash tables (DHTs) to map data to specific nodes. Each peer is assigned a portion of the keyspace, and queries are routed deterministically based on the key. This makes lookups fast and efficient, often requiring only logarithmic time relative to the number of nodes.

Example: Chord, Kademlia, Pastry. **Analogy:** A well-organized library with an index for every book. **Trend:** Widely used in decentralized storage and blockchain systems.

Diagram:

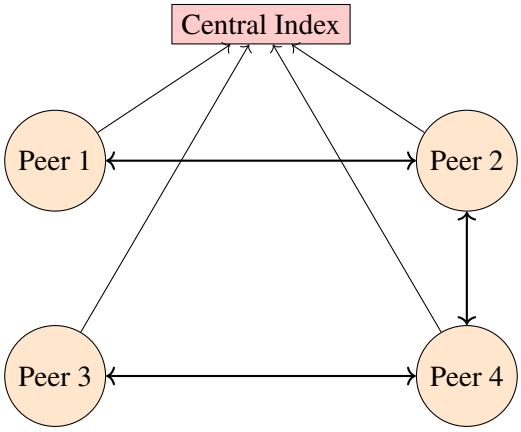


3.2.3 Hybrid P2P Architecture

Hybrid P2P architectures blend peer-to-peer principles with centralized elements to improve performance and usability. Typically, a central server helps with tasks like indexing or peer discovery, while actual data exchange happens directly between peers. This model reduces search overhead and improves scalability while retaining the decentralized nature of data transfer.

Example: Napster. **Analogy:** A matchmaking service that connects people, but the conversation happens directly. **Trend:** Common in media sharing, collaborative tools, and early P2P systems.

Diagram:



3.3 Comparison of P2P Architecture Types

Feature	Unstructured P2P	Structured P2P	Hybrid P2P
Topology	Random connections	Organized via DHT	Central index + peer links
Search Method	Flooding or random walk	Deterministic lookup	Centralized discovery, direct transfer
Efficiency	Low for rare data	High and predictable	Moderate
Fault Tolerance	High	Moderate to high	Depends on central node
Scalability	Good, but inefficient	Excellent	Good with central bottleneck risk
Example Systems	Gnutella, Freenet	Chord, Kademlia, Pastry	Napster, early Skype
Analogy	Asking everyone in a crowd	Indexed library system	Matchmaking service
Use Cases	Dynamic networks, anonymity	Decentralized storage, blockchain	Media sharing, hybrid apps

Table 3.1: Comparison of Peer-to-Peer Architecture Types

Selecting the appropriate Peer-to-Peer architecture depends on the specific goals and constraints of the system being designed. If simplicity and resilience to frequent node changes are priorities, **Unstructured P2P** is a good fit—especially for applications where data availability is high and search precision is less critical. For systems requiring efficient and predictable data lookup, such as decentralized storage or blockchain platforms, **Structured P2P** offers strong performance through organized routing and indexing. Meanwhile, **Hybrid P2P** is ideal when a balance between centralized coordination and decentralized data exchange is needed, such as in media sharing or collaborative tools. Designers must weigh factors like scalability, fault tolerance, search efficiency, and maintenance complexity to choose the model that best aligns with their application's requirements.

3.4 Architectural Patterns: Tiering vs. Layering

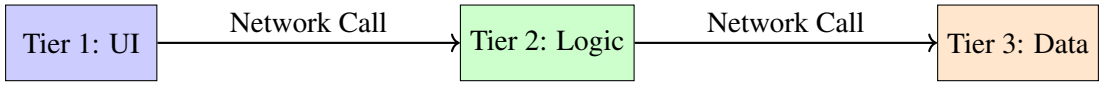
3.4.1 Tiering: Horizontal Splitting

Tiering is an architectural pattern that divides a system into distinct tiers based on functionality, with each tier deployed separately across machines or services. This horizontal splitting allows for clear separation of concerns and modular deployment. A classic example is an airline search application:

- **Tier 1:** User interface for flight search
- **Tier 2:** Application logic for filtering and processing
- **Tier 3:** Data access for retrieving flight schedules and prices

This structure enables teams to scale and maintain each tier independently. The benefits include easier maintenance and clearer responsibility boundaries. However, tiering introduces complexity in deployment and communication, and can increase latency due to network hops between tiers. It's like a relay race—each runner (tier) has a defined role, but passing the baton adds coordination overhead.

Diagram:



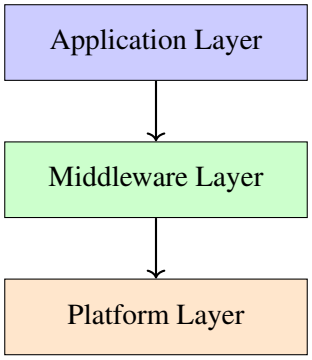
3.4.2 Layering: Vertical Organization

Layering organizes a system into stacked layers, where each layer builds on the services provided by the one below it. This vertical structure simplifies design by hiding complexity and promoting abstraction. Common layers include:

- **Applications:** End-user services like browsers or email clients
- **Middleware:** Communication tools like RPC, messaging, or service registries
- **Platform:** Operating system and hardware infrastructure

Layering is especially useful in distributed systems where abstraction helps manage complexity. For example, an application doesn't need to know how the OS schedules threads—it simply relies on the platform layer. This pattern promotes modularity and reusability, making it easier to swap or upgrade components. The analogy of a layered cake fits perfectly: each layer adds flavor and builds on the previous one, but the eater (user) only sees the top.

Diagram:



Comparison of Tiering and Layering

Aspect	**Tiering (Horizontal)**	**Layering (Vertical)**
Definition	Divides system by functional roles across physical tiers	Organizes system by logical abstraction levels
Deployment	Each tier may run on separate machines	Layers may reside on the same machine or across tiers
Example	Airline search app: UI, logic, data tiers	OS stack: application, middleware, platform
Analogy	Relay race with baton passing	Layered cake with stacked flavors
Pros	Clear separation of concerns, modular scaling	Simplifies design, hides complexity, promotes reuse
Cons	Added latency, network traffic, deployment complexity	Can obscure performance issues, harder debugging
Use Cases	Web apps, enterprise systems, client-server models	OS design, middleware platforms, layered protocols

Table 3.2: Comparison of Tiering and Layering Architectural Patterns

3.5 Excercises

Scenario 1. A distributed banking system uses RPC over UDP to process transactions. A customer reports that their transaction was processed twice. What RPC semantic might be missing, and how would you redesign the system to prevent this?

Scenario 2. You are designing a distributed file storage system. You must choose between a Master-Slave and Peer-to-Peer architecture. The system must support high availability and scale to millions of users. Which architecture would you choose and why?

Scenario 3. A video streaming platform uses a hybrid P2P model. During peak hours, users complain about slow content delivery. What architectural bottlenecks might be causing this, and how could you improve the system?

Scenario 4. A search engine backend is structured as a three-tier architecture. The application logic tier is experiencing high latency. What strategies could you use to isolate and resolve the issue?

Scenario 5. A multiplayer game uses structured P2P for matchmaking. Players report frequent disconnects and slow game starts. How does churn affect this system, and what mitigation strategies would you recommend?

Scenario 6. A university deploys a distributed learning platform using tiering. The UI tier is hosted on a separate server from the application and data tiers. Students report delays in loading course materials. What architectural adjustments could improve performance?

Scenario 7. A blockchain-based voting system uses a pure P2P architecture. How would you ensure data integrity and availability during high churn and network partitioning?

Scenario 8. An e-commerce site uses a Master-Slave model for inventory management. The master node crashes during a major sale event. What are the consequences, and how could you redesign the system to avoid this failure?

Scenario 9. A distributed chat application uses unstructured P2P. Users complain that old messages are no longer accessible. What architectural limitation is causing this, and how could you redesign the system?

Scenario 10. A cloud-based analytics platform uses layering to separate data ingestion, processing, and visualization. A bug in the processing layer causes incorrect results. How does layering help isolate and fix the issue?

Scenario 11. A peer-to-peer file sharing system uses overlay networks. Users report that downloads are slow despite good internet connections. What might be causing this, and how could overlay routing be optimized?

Scenario 12. A distributed sensor network uses structured P2P to collect environmental data. Some sensors frequently go offline. How does this affect the system, and what architectural changes could improve robustness?

Scenario 13. A travel booking system uses a three-tier architecture. The database tier is under heavy load. What caching or replication strategies could be used to reduce pressure on the data tier?

Scenario 14. A social media platform uses middleware to abstract communication between services. A developer wants to add a new messaging feature. How does middleware simplify this integration?

Scenario 15. A distributed ledger system uses hybrid P2P with central nodes for coordination. One central node becomes overloaded. What architectural redesign could distribute the load more effectively?

3.6 Answers

Answer 1. The system likely lacks at-most-once semantics. Redesign using unique transaction IDs and persistent storage to detect duplicates and ensure idempotency.

Answer 2. Peer-to-Peer is preferable due to its decentralized nature, eliminating SPOF and enabling horizontal scalability across millions of users.

Answer 3. Bottlenecks may stem from overloaded central servers. Introduce more super-peers or decentralize coordination to reduce load and improve throughput.

Answer 4. Use profiling tools to monitor latency sources. Consider caching, load balancing, or scaling the application logic tier horizontally.

Answer 5. Churn disrupts routing and peer availability. Use redundancy, heartbeat protocols, and backup peers to maintain stability.

Answer 6. Introduce caching at the UI tier, optimize inter-tier communication, and consider edge servers to reduce latency.

Answer 7. Use replication, consensus algorithms (e.g., Raft), and distributed hash tables to maintain integrity and availability under churn.

Answer 8. The master node is a SPOF. Use leader election protocols or replicate the master role across multiple nodes for fault tolerance.

Answer 9. Unstructured P2P lacks guaranteed data persistence. Introduce structured indexing or hybrid models with backup nodes for message retention.

Answer 10. Layering isolates bugs to specific layers. Fixes in the processing layer won't affect ingestion or visualization, simplifying debugging.

Answer 11. Overlay routing may not align with physical paths. Optimize with proximity-aware routing or dynamic peer selection.

Answer 12. Offline sensors break routing paths. Use redundant paths, backup nodes, and dynamic reconfiguration to maintain data flow.

Answer 13. Use read replicas, caching layers (e.g., Redis), and load balancing to reduce direct pressure on the database tier.

Answer 14. Middleware abstracts communication, allowing the new messaging service to plug into existing APIs without modifying lower layers.

Answer 15. Distribute coordination across multiple central nodes or promote capable peers to super-peer roles to balance load.

3.7 Multiple-Choice Quiz

Q1. Which of the following best explains why fault-tolerance is still necessary in RPC layered over TCP?

a. TCP guarantees application-level correctness.
b. TCP handles all types of failures, including server crashes.
c. TCP only ensures reliable transmission, not end-to-end correctness.
d. TCP eliminates the need for idempotency.

Q2. In a Master-Slave architecture, what is the primary drawback when scaling out the system?

a. Increased latency due to peer routing.
b. The master becomes a performance bottleneck.
c. Lack of coordination among slaves.
d. Difficulty in maintaining consistency.

Q3. Why might a designer prefer a Peer-to-Peer architecture for a content-sharing platform?

a. It simplifies decision-making.
b. It avoids single points of failure and scales well.
c. It centralizes control for better performance.
d. It reduces metadata overhead.

Q4. What is a key challenge in unstructured P2P networks?
a. Maintaining consistent hashing.
b. Handling metadata replication.
c. Locating data efficiently.
d. Preventing centralization.

Q5. Which architectural pattern best supports modular debugging and abstraction?
a. Tiering
b. Layering
c. Master-Slave
d. Hybrid P2P

Q6. What is a consequence of high churn in structured P2P systems?
a. Improved robustness.
b. Simplified routing.
c. Frequent metadata updates and instability.
d. Reduced overlay complexity.

Q7. Why might unpopular data disappear in a pure P2P system?
a. It is stored on central servers.
b. Peers stop sharing it, making it inaccessible.
c. It is replicated across all nodes.
d. It is cached permanently.

Q8. What is the main benefit of hybrid P2P architectures?
a. Eliminates all central coordination.
b. Combines scalability with centralized discovery.
c. Reduces the need for overlay networks.
d. Guarantees data consistency.

Q9. In a three-tier architecture, which tier typically handles ranking and business logic?
a. Presentation tier
b. Application tier
c. Data tier
d. Middleware tier

Q10. What is a disadvantage of layering in distributed systems?
a. Increased modularity
b. Simplified abstraction
c. Added latency due to vertical communication
d. Easier debugging

Q11. Which of the following best describes the role of middleware?
a. It manages hardware resources.
b. It provides low-level networking protocols.
c. It abstracts heterogeneity and simplifies programming.
d. It stores application data.

Q12. What architectural strategy helps reduce load on a database tier?
a. Adding more presentation servers
b. Using structured P2P
c. Implementing caching and replication
d. Switching to UDP

Q13. Why is idempotency important in RPC design?
a. It ensures procedures are executed only once.
b. It allows safe re-execution without side effects.
c. It eliminates the need for fault-tolerance.
d. It improves network throughput.

Q14. What does tiering primarily organize in a distributed system?

a. Vertical abstraction of services
b. Horizontal placement of functionality across servers
c. Metadata replication
d. Peer routing

Q15. Which layer in a layered architecture typically interacts directly with hardware?

a. Application layer
b. Middleware layer
c. Platform layer
d. Presentation layer

4. Naming Systems in Distributed Systems

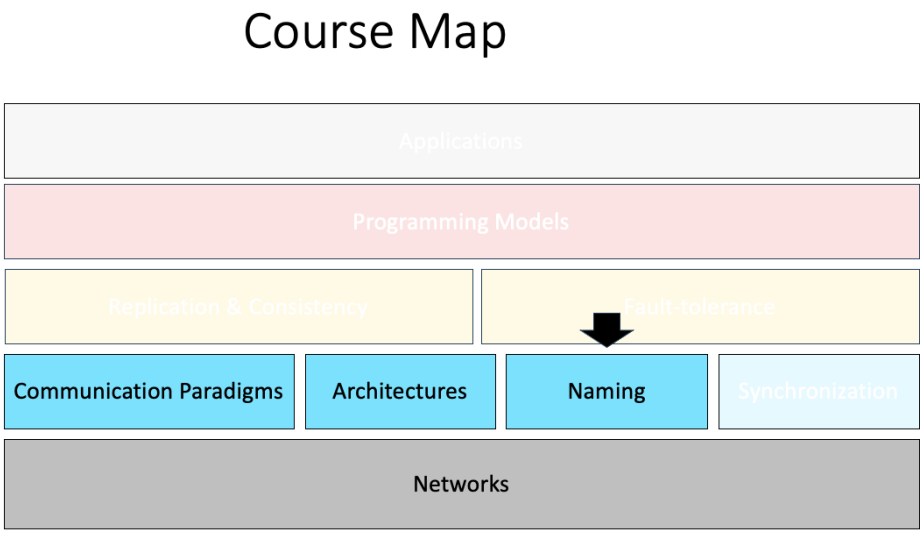


Figure 4.1: Naming

4.1 Introduction to Naming

Key Idea

Naming is a foundational concept in distributed systems. It enables entities—such as files, processes, services, or devices—to be identified and located across a network of machines. Unlike centralized systems, distributed environments span multiple nodes, networks, and administrative domains, making naming both more powerful and more complex.

Why Naming Matters

Naming provides a level of abstraction that decouples the identity of an entity from its physical location. This abstraction is essential for:

- **Mobility:** Entities can move across machines without changing their name.

- **Replication:** Multiple copies of an entity can share the same name.
- **Fault Tolerance:** Systems can redirect requests to backup entities if the primary fails.
- **Abstraction**: Hiding low-level details behind symbolic names.
- **Modularity**: Allowing components to be reused and independently managed.
- **Communication**: Enabling interaction between hardware and software layers.

Think of naming as the "address book" of a distributed system. Without it, every interaction would require low-level knowledge of where things are and how to reach them—an impossible task at scale.

Types of References

To understand naming, we must distinguish between three types of references:

Reference Type	Description	Example
Name	Human-readable or system-generated label	`earth.html`
Address	Location-dependent reference	`55.55.55.55:8888`
Identifier	Unique, persistent reference	`02:60:8c:02:b0:5a`

Table 4.1: Types of References in Distributed Systems

> **Note**
>
> A robust naming system allows you to use a name like `earth.html` and resolve it to an address and identifier behind the scenes.

Real-World Analogy

Imagine a university campus:

- `Room 101` is a name for a physical space.
- `Professor ID 12345` is a unique identifier.
- `"Physics Lab"` is a symbolic name used by students.

Each serves a different purpose—just like names in computer systems.

Name Resolution

Name resolution is the act of translating a name into a usable reference. Consider the following example:

```
http://www.cdk5.net:8888/WebExamples/earth.html
```

This involves several layers of resolution:

1. **DNS Lookup:** `www.cdk5.net` → IP address `55.55.55.55`
2. **Port Resolution:** `:8888` → Specific service on the host
3. **Path Resolution:** `/WebExamples/earth.html` → File location
4. **MAC Resolution:** IP → MAC address `02:60:8c:02:b0:5a` (via ARP)

Each step abstracts away complexity, allowing users to interact with high-level names while the system handles the details.

Diagram: Name Resolution Pipeline

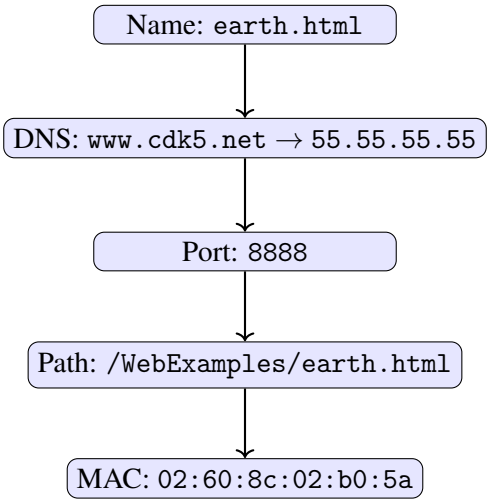


Figure 4.2: Name Resolution Pipeline in Distributed Systems

Exercise: Trace the Resolution

Given the URL `http://files.example.org:8080/data/report.pdf`, identify:

- The name
- The address
- The identifier (assume MAC resolution is available)
- The steps required to resolve the full reference

4.2 Naming Systems in Distributed Systems

Distributed systems rely on naming schemes to identify and locate entities such as files, devices, or services. Three primary naming models are commonly used: **Flat Naming**, **Structured Naming**, and **Attribute-Based Naming**.

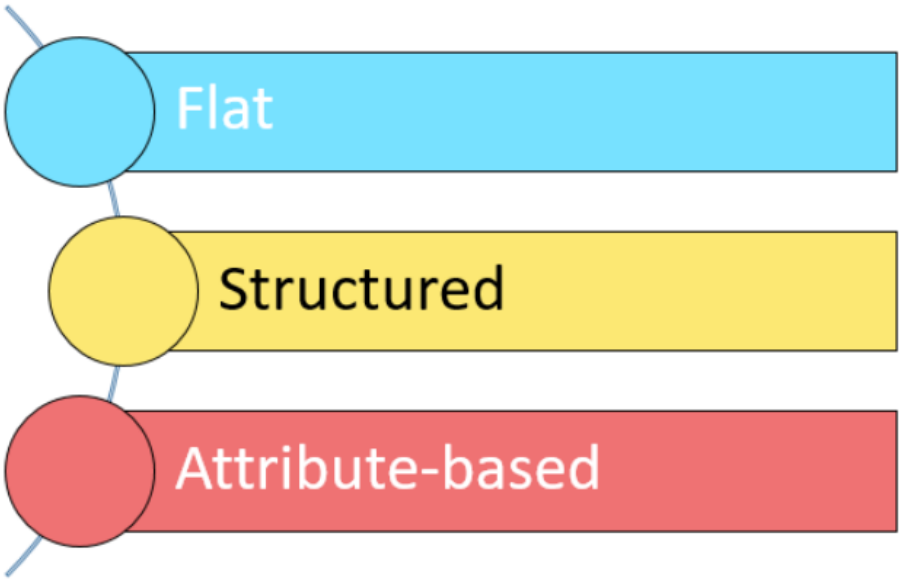


Figure 4.3: Naming Systems Types

1. Flat Naming

Flat naming assigns each entity a globally unique identifier without embedding any structural or location-based information. These identifiers are typically opaque and non-human-readable.

- **Example:** MAC addresses (e.g., `02:60:8c:02:b0:5a`) or UUIDs.
- **Advantages:** Supports mobility and location independence; ideal for peer-to-peer and mobile systems.
- **Challenges:** Requires auxiliary mechanisms (e.g., broadcasting, forwarding pointers) for name resolution.

Name resolution is the process of mapping a name to the entity's current location or address. This is essential in flat naming systems where names lack embedded location data.

Broadcasting

Mechanism: The system sends a query to all nodes asking who holds the name.
Pros: Simple, decentralized
Cons: Inefficient in large networks; high traffic
Analogy: Shouting a name in a crowded room

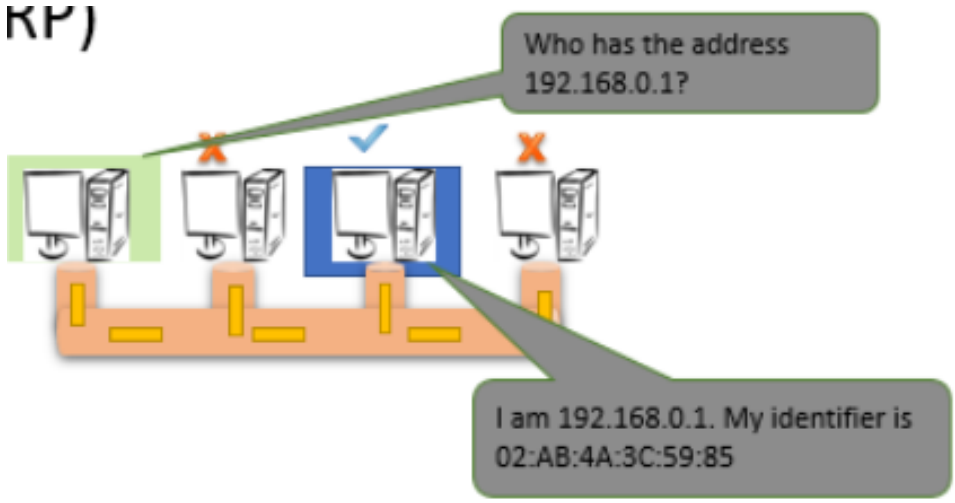


Figure 4.4: Broadcasting Name Resolution

Forwarding Pointers

Mechanism: When an entity moves, it leaves a pointer at its old location.
Pros: Efficient for mobile entities
Cons: Pointer chains can grow long; requires cleanup
Analogy: Leaving sticky notes at each previous location

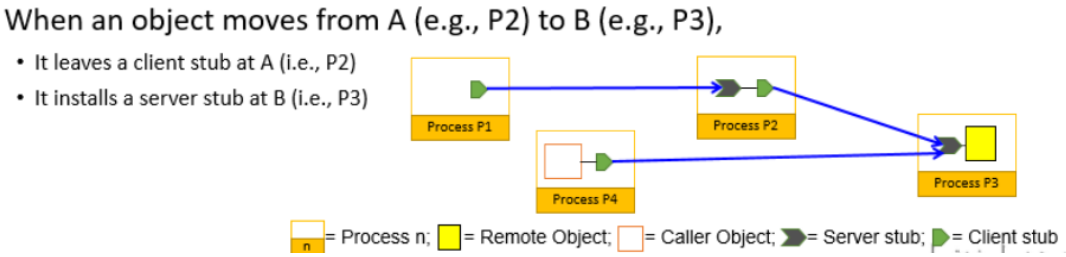


Figure 4.5: Forward Pointer Name Resolution

Home-Based Name Resolution Process

In home-based name resolution, each mobile entity is assigned a static "home node" that acts as a central authority for tracking its current location. When the entity moves to a new network or host, it updates its home node with its new foreign address. The following steps outline the sequence of communication in a home-based name resolution system for mobile IP:

Step 1: Location Update
The mobile entity moves to a new network and registers its current *foreign address* with its designated *home node*.

Step 2: Initial Packet Transmission
A client sends a packet to the mobile entity's *home address*, unaware that the entity has relocated.

Step 3: Forwarding and Notification

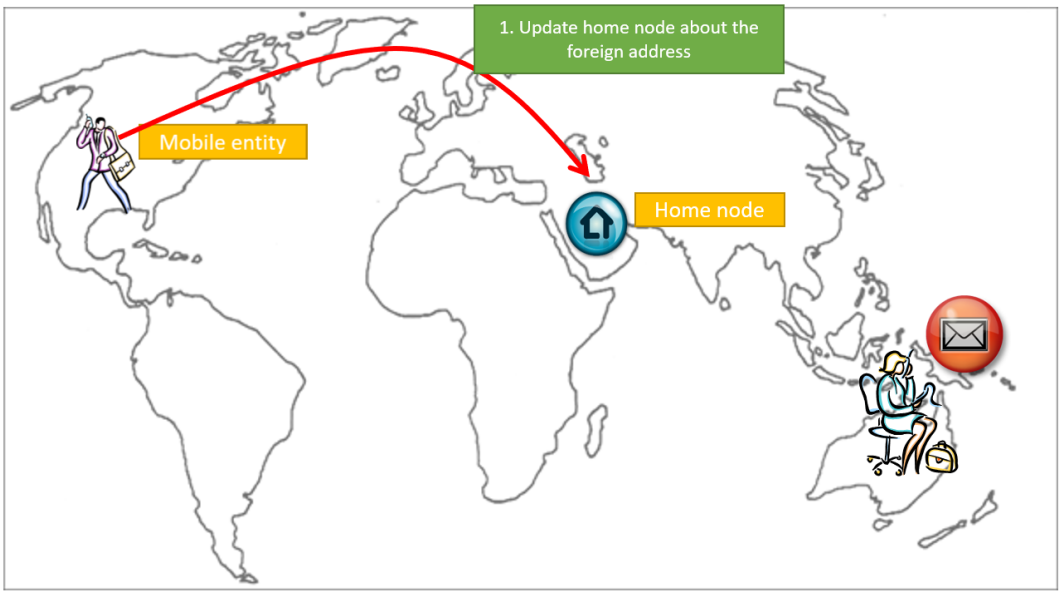


Figure 4.6: Step 1: Location Update Forward Pointer Name Resolution

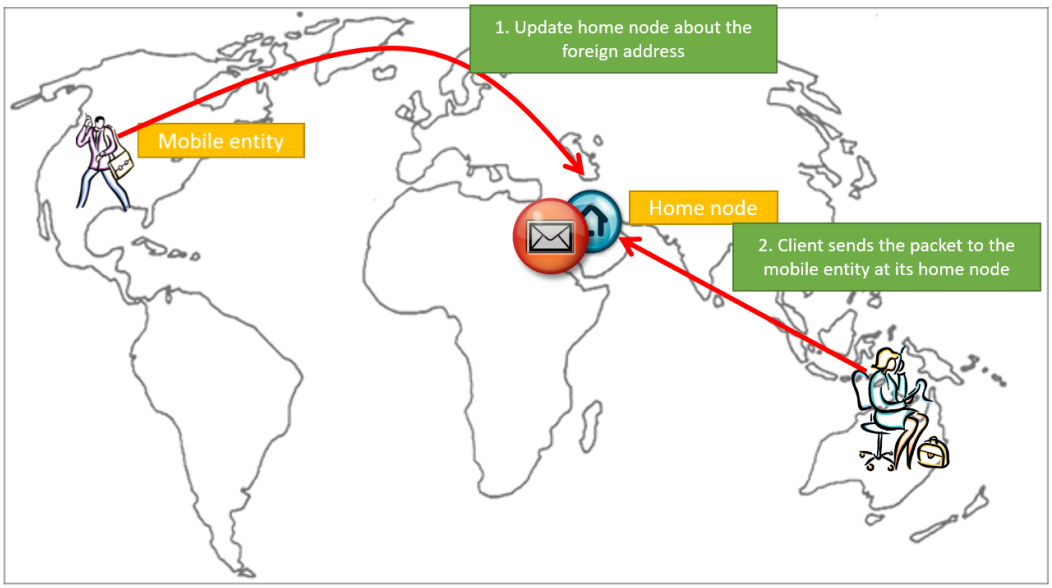


Figure 4.7: Step 2: Initial Packet Forward Pointer Name Resolution

- The home node intercepts the packet and forwards it to the mobile entity's *foreign address*.
- Simultaneously, the home node replies to the client with the mobile entity's updated IP address.

Step 4: Direct Communication
The client uses the foreign address to send all subsequent packets directly to the mobile entity, bypassing the home node.

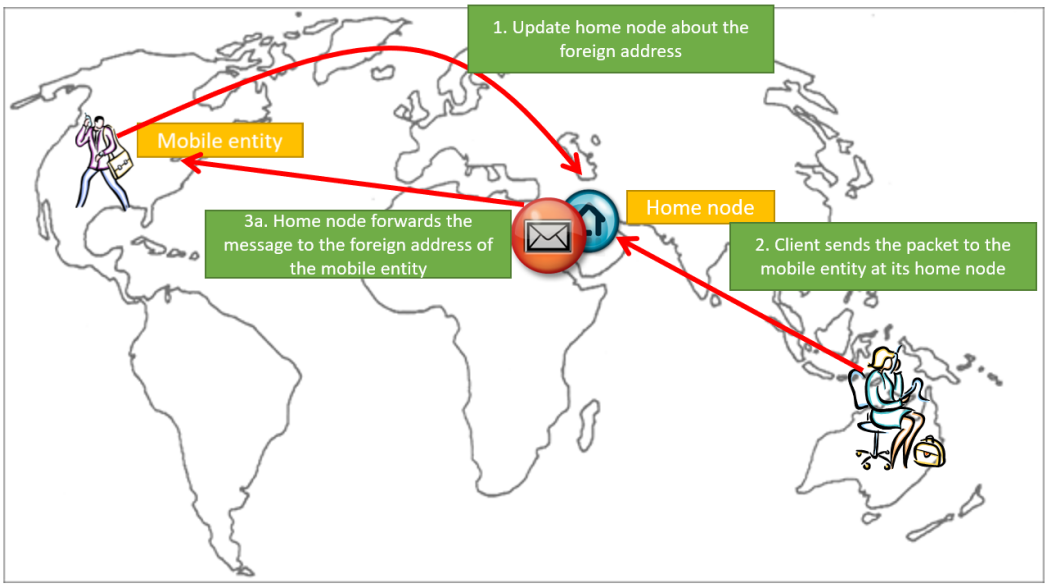


Figure 4.8: Step 3: Forwarding and Notification Forward Pointer Name Resolution

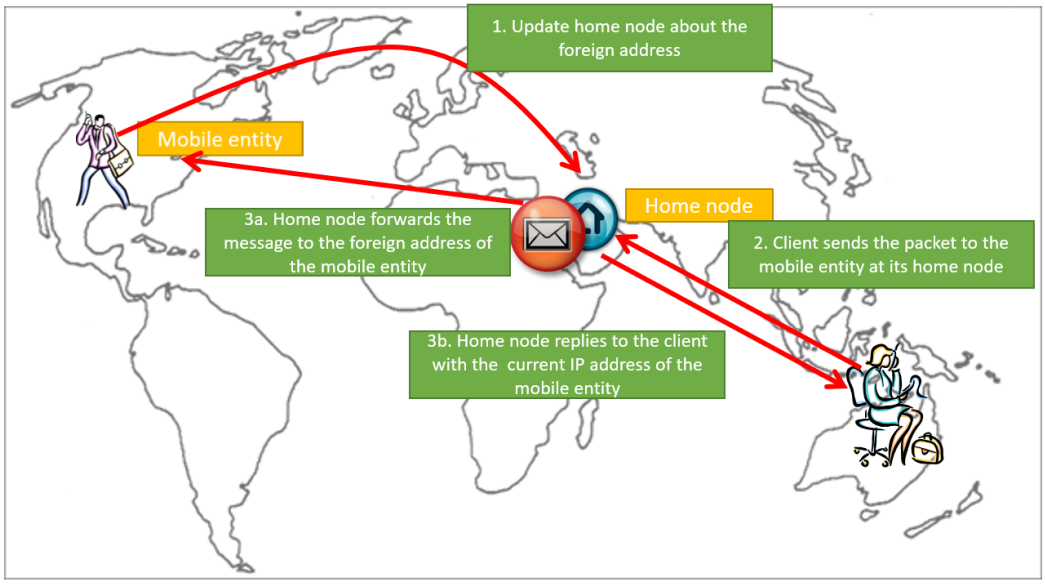


Figure 4.9: Step 4: Forwarding and Notification Forward Pointer Name Resolution

Distributed Hash Table DHT

A **Distributed Hash Table (DHT)** is a decentralized data structure that maps keys to values across a network of nodes. It enables scalable and fault-tolerant name resolution in distributed systems.

Each node is responsible for a portion of the keyspace, and queries are routed efficiently using structured overlays.

Key Properties

- **Scalability:** Supports millions of nodes and keys.
- **Decentralization:** No central authority or single point of failure.
- **Efficiency:** Lookup operations typically require $\mathcal{O}(\log n)$ hops.

- **Fault Tolerance:** Robust against node joins and failures.

Chord Protocol

Chord is a well-known DHT protocol that organizes nodes in a circular keyspace of size 2^m, where m is the number of bits in the hash function output (a ring). Each node maintains a finger table with pointers to other nodes at exponentially increasing distances.

- Each node has an ID: $n_i = \text{hash}(IP_i)$
- Each key k is stored at the node whose ID is the first to equal or succeed k in the ring.
- Each node maintains a **finger table** for efficient routing.

Finger Tables

- Store pointers at exponential distances
- Time complexity: $O(\log n)$

Entry	Finger Table FTp[i]	Meaning
1	succ(p + 2^0)	Next node
2	succ(p + 2^1)	2 hops ahead
⋮	⋮	⋮

Table 4.2: Chord Finger Table Entries

For node n, the i-th entry in its finger table is:

$$FT_n[i] = \text{successor}(n + 2^{i-1}) \mod 2^m$$

This allows Chord to resolve lookups in $\mathcal{O}(\log n)$ time.

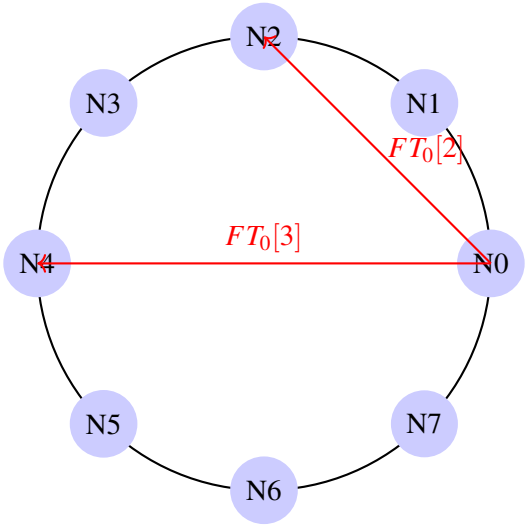


Figure 4.10: Chord Ring with Finger Table Routing

Naïve Resolution Algorithm

The **Naïve Key Resolution Algorithm** is a simple method used in Distributed Hash Tables (DHTs) to locate the node responsible for a given key. It involves sequential traversal of nodes in the overlay network until the target is found.

Algorithm Steps

Given a key k and a starting node n:

1. Check if k lies between n and its successor.
2. If yes, the successor is responsible for k.
3. If not, forward the query to the successor.
4. Repeat until the responsible node is located.

Time Complexity

- Worst-case lookup time: $\mathcal{O}(n)$ hops.
- Not scalable for large networks.

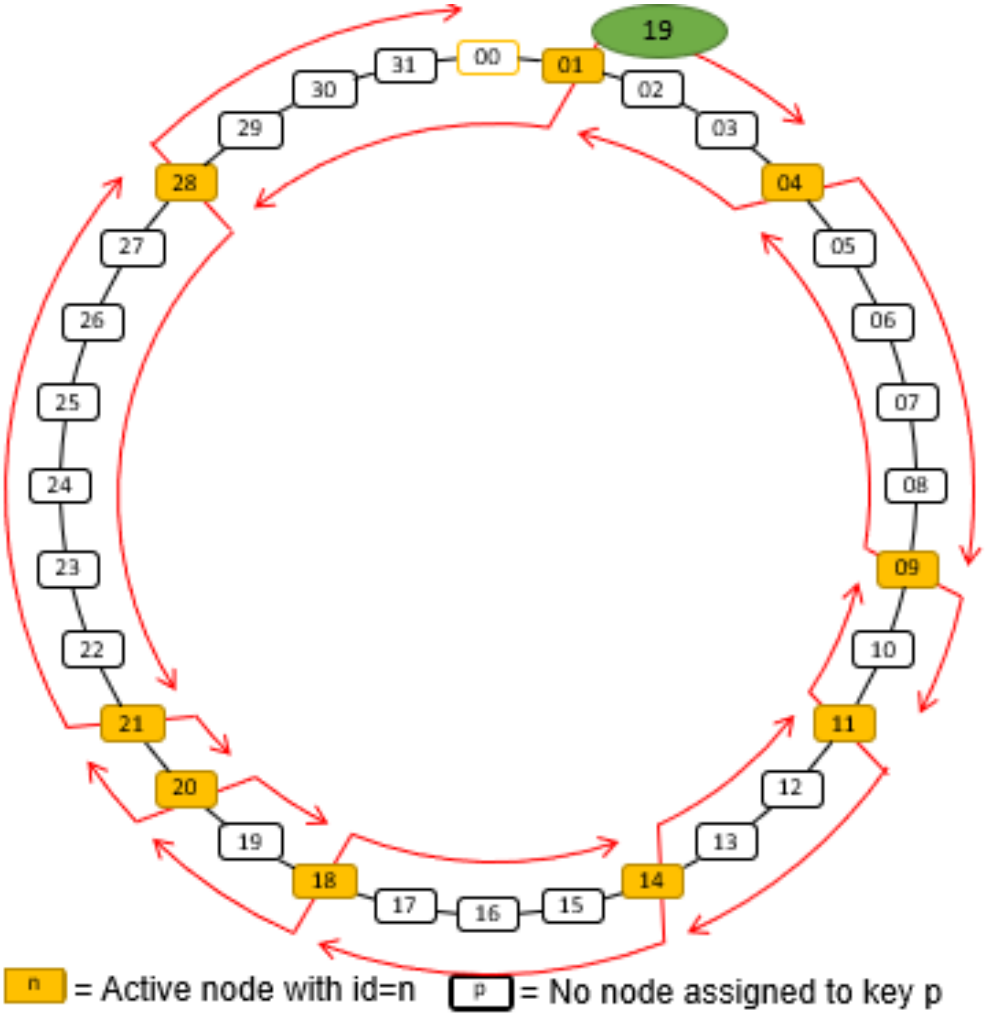


Figure 4.11: Naiive Resolution Algorithm

Chord Key Resolution

Chord improves key resolution by reducing the time complexity to $\mathcal{O}(\log n)$, where n is the number of nodes in the system. All nodes are arranged in a logical ring based on their hashed identifiers.

Finger Table

Each node p maintains a **Finger Table** FT_p with at most m entries, where m is the number of bits in the identifier space.

$$FT_p[i] = \text{successor}(p + 2^{i-1}) \mod 2^m$$

Note: The entries in FT_p increase exponentially with i.

Key Resolution Procedure

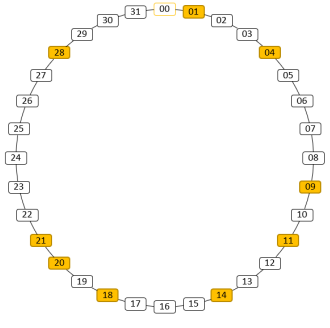
When node p receives a request to resolve key k, it performs the following steps:

- Find the index j such that:

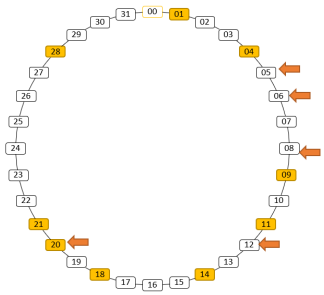
$$FT_p[j] \leq k < FT_p[j+1]$$

- Forward the request to node $q = FT_p[j]$.
- If $k > FT_p[m]$, then forward the request to $FT_p[m]$.
- If $k < FT_p[1]$, then forward the request to $FT_p[1]$.

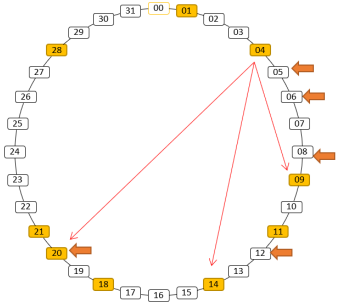
This routing strategy ensures that each lookup moves closer to the target key exponentially, resulting in efficient resolution across the ring.



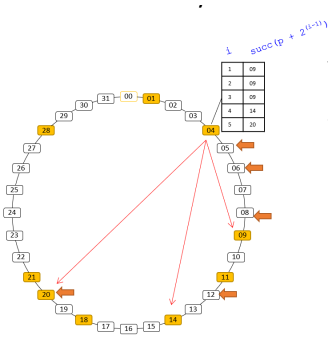
(a) 1st step



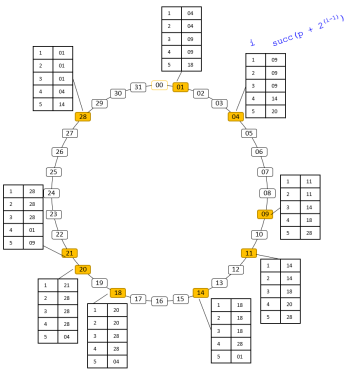
(b) 2nd step



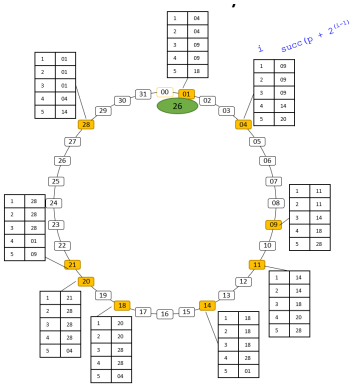
(c) 3rd step



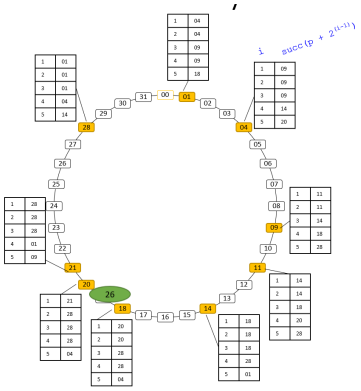
(d) 4th step



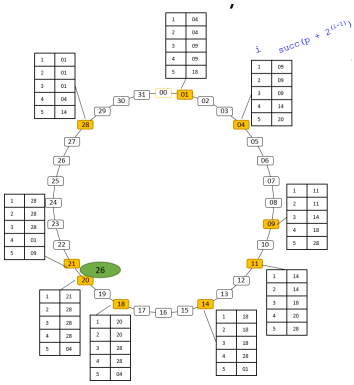
(e) 5th step



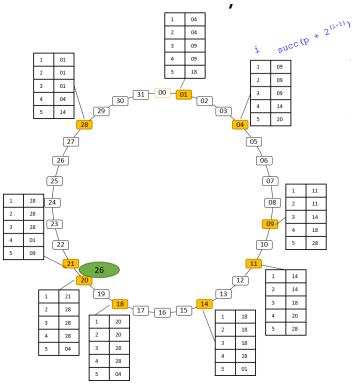
(f) 6th step



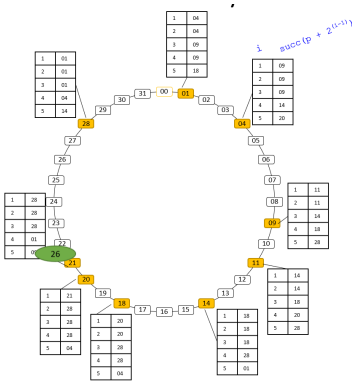
(g) 7th step



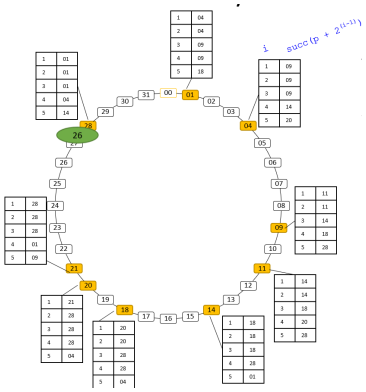
(h) 8th step



(i) 8th step



(j) 9th step



(k) 10th step

Figure 4.12: Chord Key Resolution Procedure Steps

2. Structured Naming and Naming Graphs

Structured naming refers to the use of hierarchical, human-readable names that reflect the organization of resources. These names are typically composed of segments separated by delimiters (e.g., slashes in file paths or dots in domain names). Each segment represents a contextual level, allowing users to navigate through a logical hierarchy.

In distributed systems, structured names are resolved using naming graphs—directed graphs where nodes represent entities and edges are labeled with name segments. Leaf nodes correspond to actual resources (e.g., files), while directory nodes serve as intermediaries that point to other nodes. This graph-based model supports efficient traversal and resolution of names, enabling systems to locate resources based on their hierarchical structure.

Examples of Structured Naming

File System Context In the context of a file system, the path:

```
/home/userid/work/dist-systems/naming.txt
```

is the structured name of the file `naming.txt`. Each segment in the path represents a directory level, forming a hierarchical structure that leads to the target file.

Web Context In the context of a website, the URL:

```
https://ci.suez.edu/ggg/15440-f23/
```

is the structured name of the `15440` course webpage. The URL segments reflect the domain, user directory, and course folder, forming a navigable hierarchy within the web server's namespace.

Structured names are organized into **name spaces**, which define the context and rules for interpreting names. A name space can be modeled as a **directed graph**, commonly referred to as a *naming graph*, consisting of two types of nodes:

Leaf Nodes

- Represent entities such as files, webpages, or services.
- Typically store:
  - **Addresses** (e.g., IP addresses in DNS),
  - **States** (e.g., contents of files in file systems),
  - **Absolute path names** to other nodes or resources.

Directory Nodes

- Refer to other leaf or directory nodes within the graph.
- Each outgoing edge is represented as a pair:

```
(edge label, destination node identifier)
```

- These edges define the traversal paths used during name resolution.

Unix File System as a Naming Graph

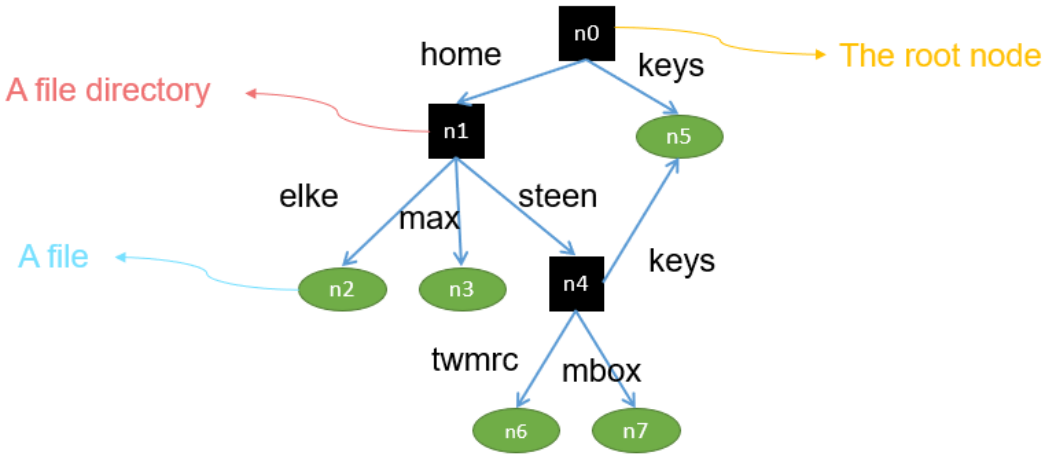


Figure 4.13: Unix Naming Graph Example

The Unix file system exemplifies structured naming through its use of inodes and data blocks. At the physical level, the file system is composed of several key components:

- **Boot Block**: is a special block of data and instructions that are automatically loaded into main memory when the system is booted , It is used to load the OS into main memory

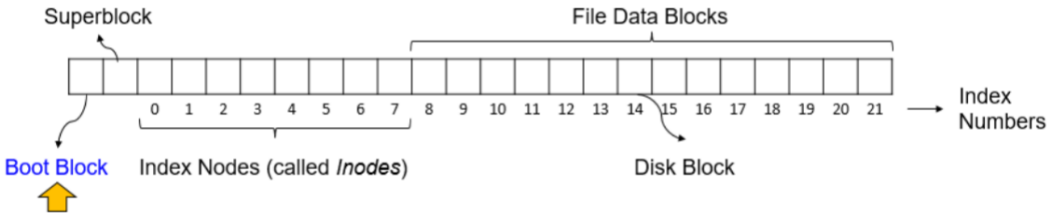


Figure 4.14: Boot Block

- **Superblock**: contains information on the entire file system (e.g., its size, which blocks on disk are not yet allocated, which Inodes are not yet used, etc.,)

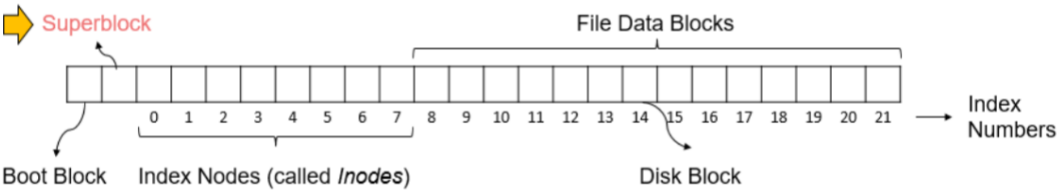


Figure 4.15: Super Block

- **Inodes**: The Inodes are referred to by index numbers, starting at zero, which is reserved for the Inode representing the root directory . Each Inode contains information on where the data of its associated file can be found on disk . Besides, an Inode contains information on its owner, time of creation and last modification, protection, etc.,

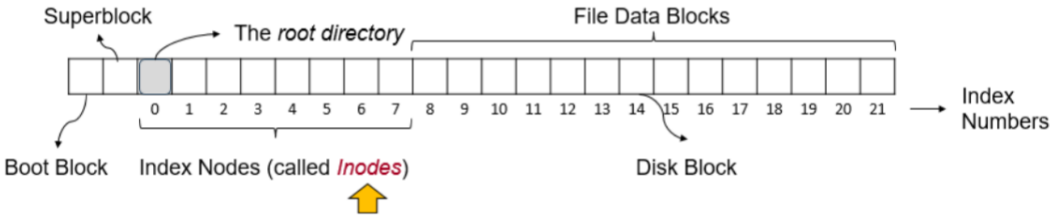


Figure 4.16: Inode

- **Data Blocks**: Hold the actual contents of files.

Name resolution in Unix involves traversing the naming graph by following directory entries. For example, resolving the path `/home/steen/mbox` requires reading the root inode, locating the "home" directory, then "steen," and finally the "mbox" file. Each step involves accessing the corresponding inode and interpreting its directory entries to continue traversal.

Linking Mechanisms: Hard and Symbolic Links

Unix supports two types of links that allow multiple names to refer to the same resource: hard links and symbolic links.

Hard links create multiple directory entries that point to the same inode. This means that different names can refer to the same file, and changes made through one name are reflected in all others. However, hard links are restricted to the same file system and cannot form cycles.

Symbolic links, on the other hand, store the path to another file as their content. When accessed, the system reads the symbolic link and resolves the referenced path. Symbolic links can span file systems and support more flexible referencing, but they introduce an additional layer of indirection during resolution.

Mounting and Distributed Name Spaces

Mounting is a technique used to integrate multiple name spaces into a single unified hierarchy. In distributed systems, mounting allows directories from remote machines to appear as part of the local file system. This is commonly used in networked file systems like NFS.

For example, the path `/remote/vu/home/steen/mbox` may refer to a file stored on a remote server, but it is accessed as if it were part of the local directory structure. Mounting provides transparency and location independence, enabling users to interact with distributed resources seamlessly.

Distributing the Naming Graph

In large-scale systems, the naming graph itself may be distributed across multiple machines. A common approach is to use a master-worker model, where the master node stores the root and key directories, while worker nodes manage subdirectories and files. This distribution improves scalability and fault tolerance, but it also introduces challenges in consistency and coordination.

Alternative models, such as peer-to-peer distribution, offer decentralized control but require more complex resolution protocols. Understanding these models is essential for designing robust and efficient distributed naming systems.

7. Attribute-Based Naming

Attribute-based naming shifts the focus from fixed hierarchical names to descriptive attributes. Instead of specifying a path, users query for resources based on properties such as location, organization, or role.

This approach is exemplified by systems like LDAP (Lightweight Directory Access Protocol), which organize records into a Directory Information Tree (DIT). Each record consists of attribute-value pairs and is uniquely identified by a Distinguished Name. Queries are expressed using logical filters, such as `&(C=NL)(O=Vrije Universiteit)(CN=Main server)`, which match records based on specified attributes.

Attribute-based naming supports flexible and semantic resource discovery, making it ideal for environments where resources are dynamic or loosely structured.

8. Summary and Comparison

To consolidate the concepts, here's a comparative overview of naming types:

Naming Type	**Mechanism**	**Example**
Flat Naming	Unique identifiers, resolved via DHT	MAC address, UUID
Structured Naming	Hierarchical names, resolved via graphs	File paths, URLs
Attribute-Based Naming	Descriptive queries, attribute matching	LDAP, service directories

Each naming approach serves different needs in distributed systems, balancing simplicity, flexibility, and scalability.

II Part Two Title

5. Mathematics

5.1 Theorems

5.1.1 Several equations

This is a theorem consisting of several equations.

Theorem 5.1 — Name of the theorem. In $E = \mathbb{R}^n$ all norms are equivalent. It has the properties:

$$\big|\,||\mathbf{x}|| - ||\mathbf{y}||\,\big| \leq ||\mathbf{x}-\mathbf{y}|| \tag{5.1}$$

$$||\sum_{i=1}^{n}\mathbf{x}_i|| \leq \sum_{i=1}^{n}||\mathbf{x}_i|| \quad \text{where } n \text{ is a finite integer} \tag{5.2}$$

5.1.2 Single Line

This is a theorem consisting of just one line.

Theorem 5.2 A set $\mathscr{D}(G)$ in dense in $L^2(G)$, $|\cdot|_0$.

5.2 Definitions

A definition can be mathematical or it could define a concept.

Definition 5.1 — Definition name. Given a vector space E, a norm on E is an application, denoted $||\cdot||$, E in $\mathbb{R}^+ = [0,+\infty[$ such that:

$$||\mathbf{x}|| = 0 \;\Rightarrow\; \mathbf{x} = \mathbf{0} \tag{5.3}$$

$$||\lambda\mathbf{x}|| = |\lambda| \cdot ||\mathbf{x}|| \tag{5.4}$$

$$||\mathbf{x}+\mathbf{y}|| \leq ||\mathbf{x}|| + ||\mathbf{y}|| \tag{5.5}$$

5.3 Notations

■ **Notation 5.1** Given an open subset G of $\mathbb{R}^n$, the set of functions φ are:

1. Bounded support G;
2. Infinitely differentiable;

a vector space is denoted by $\mathscr{D}(G)$.

5.4 Remarks

This is an example of a remark.

R The concepts presented here are now in conventional employment in mathematics. Vector spaces are taken over the field $\mathbb{K} = \mathbb{R}$, however, established properties are easily extended to $\mathbb{K} = \mathbb{C}$.

5.5 Corollaries

Corollary 5.1 — Corollary name. The concepts presented here are now in conventional employment in mathematics. Vector spaces are taken over the field $\mathbb{K} = \mathbb{R}$, however, established properties are easily extended to $\mathbb{K} = \mathbb{C}$.

5.6 Propositions

5.6.1 Several equations

Proposition 5.1 — Proposition name. It has the properties:

$$\big|\,||\mathbf{x}|| - ||\mathbf{y}||\,\big| \leq ||\mathbf{x}-\mathbf{y}|| \tag{5.6}$$

$$||\sum_{i=1}^{n} \mathbf{x}_i|| \leq \sum_{i=1}^{n} ||\mathbf{x}_i|| \quad \text{where } n \text{ is a finite integer} \tag{5.7}$$

5.6.2 Single Line

Proposition 5.2 Let $f, g \in L^2(G)$; if $\forall \varphi \in \mathscr{D}(G)$, $(f,\varphi)_0 = (g,\varphi)_0$ then $f = g$.

5.7 Examples

5.7.1 Equation Example

■ **Example 5.1** Let $G = \{x \in \mathbb{R}^2 : |x| < 3\}$ and denoted by: $x^0 = (1,1)$; consider the function:

$$f(x) = \begin{cases} \mathrm{e}^{|x|} & \text{si } |x - x^0| \leq 1/2 \\ 0 & \text{si } |x - x^0| > 1/2 \end{cases} \tag{5.8}$$

The function f has bounded support, we can take $A = \{x \in \mathbb{R}^2 : |x - x^0| \leq 1/2 + \varepsilon\}$ for all $\varepsilon \in]0; 5/2 - \sqrt{2}[$. ■

5.7.2 Text Example

■ **Example 5.2 — Example name.** Aliquam arcu turpis, ultrices sed luctus ac, vehicula id metus. Morbi eu feugiat velit, et tempus augue. Proin ac mattis tortor. Donec tincidunt, ante rhoncus luctus semper, arcu lorem lobortis justo, nec convallis ante quam quis lectus. Aenean tincidunt sodales massa, et hendrerit tellus mattis ac. Sed non pretium nibh. Donec cursus maximus luctus. Vivamus lobortis eros et massa porta porttitor. ■

5.8 Exercises

Exercise 5.1 This is a good place to ask a question to test learning progress or further cement ideas into students' minds. ■

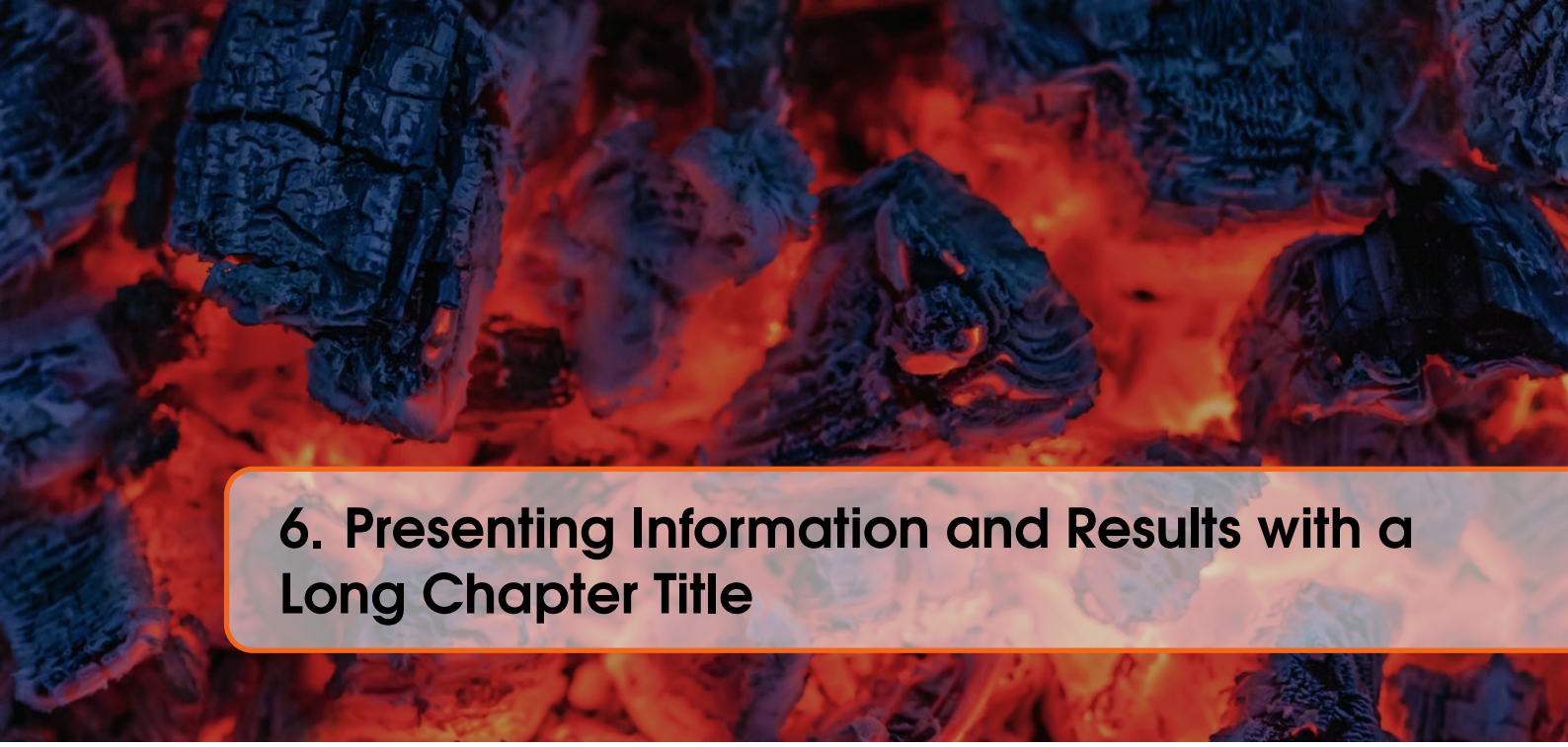
5.9 Problems

Problem 5.1 What is the average airspeed velocity of an unladen swallow?

5.10 Vocabulary

Define a word to improve a students' vocabulary.

- **Vocabulary 5.1 — Word.** Definition of word.



6. Presenting Information and Results with a Long Chapter Title

6.1 Table

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent porttitor arcu luctus, imperdiet urna iaculis, mattis eros. Pellentesque iaculis odio vel nisl ullamcorper, nec faucibus ipsum molestie. Sed dictum nisl non aliquet porttitor. Etiam vulputate arcu dignissim, finibus sem et, viverra nisl. Aenean luctus congue massa, ut laoreet metus ornare in. Nunc fermentum nisi imperdiet lectus tincidunt vestibulum at ac elit. Nulla mattis nisl eu malesuada suscipit.

Treatments	Response 1	Response 2
Treatment 1	0.0003262	0.562
Treatment 2	0.0015681	0.910
Treatment 3	0.0009271	0.296

Table 6.1: Table caption.

Referencing Table 6.1 in-text using its label.

6.2 Figure

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent porttitor arcu luctus, imperdiet urna iaculis, mattis eros. Pellentesque iaculis odio vel nisl ullamcorper, nec faucibus ipsum molestie. Sed dictum nisl non aliquet porttitor. Etiam vulputate arcu dignissim, finibus sem et, viverra nisl. Aenean luctus congue massa, ut laoreet metus ornare in. Nunc fermentum nisi imperdiet lectus tincidunt vestibulum at ac elit. Nulla mattis nisl eu malesuada suscipit.

Figure 6.1: Figure caption.

Referencing Figure 6.1 in-text using its label.

Treatments	Response 1	Response 2
Treatment 1	0.0003262	0.562
Treatment 2	0.0015681	0.910
Treatment 3	0.0009271	0.296

Table 6.2: Floating table.

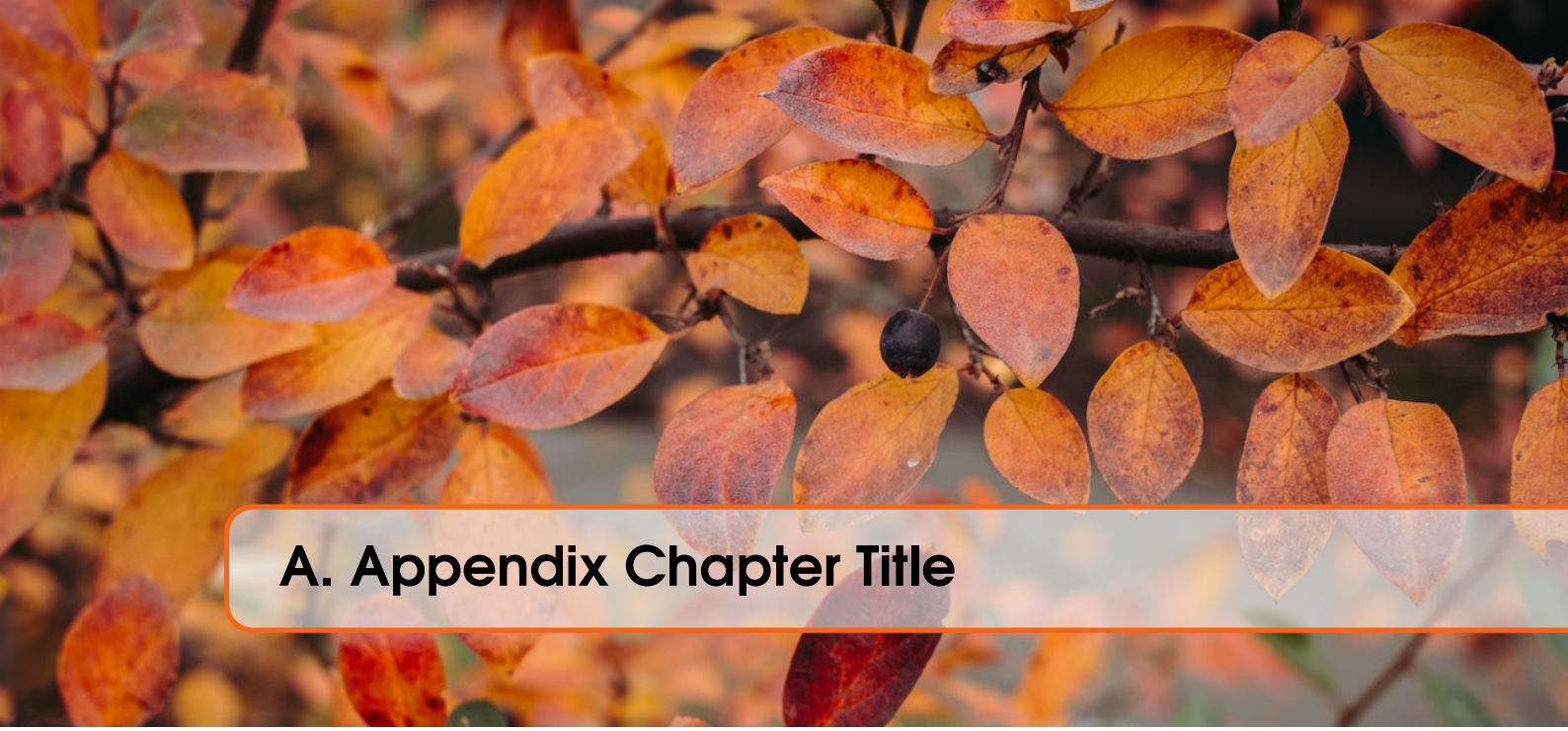
Figure 6.2: Floating figure.

Bibliography

Articles

Books

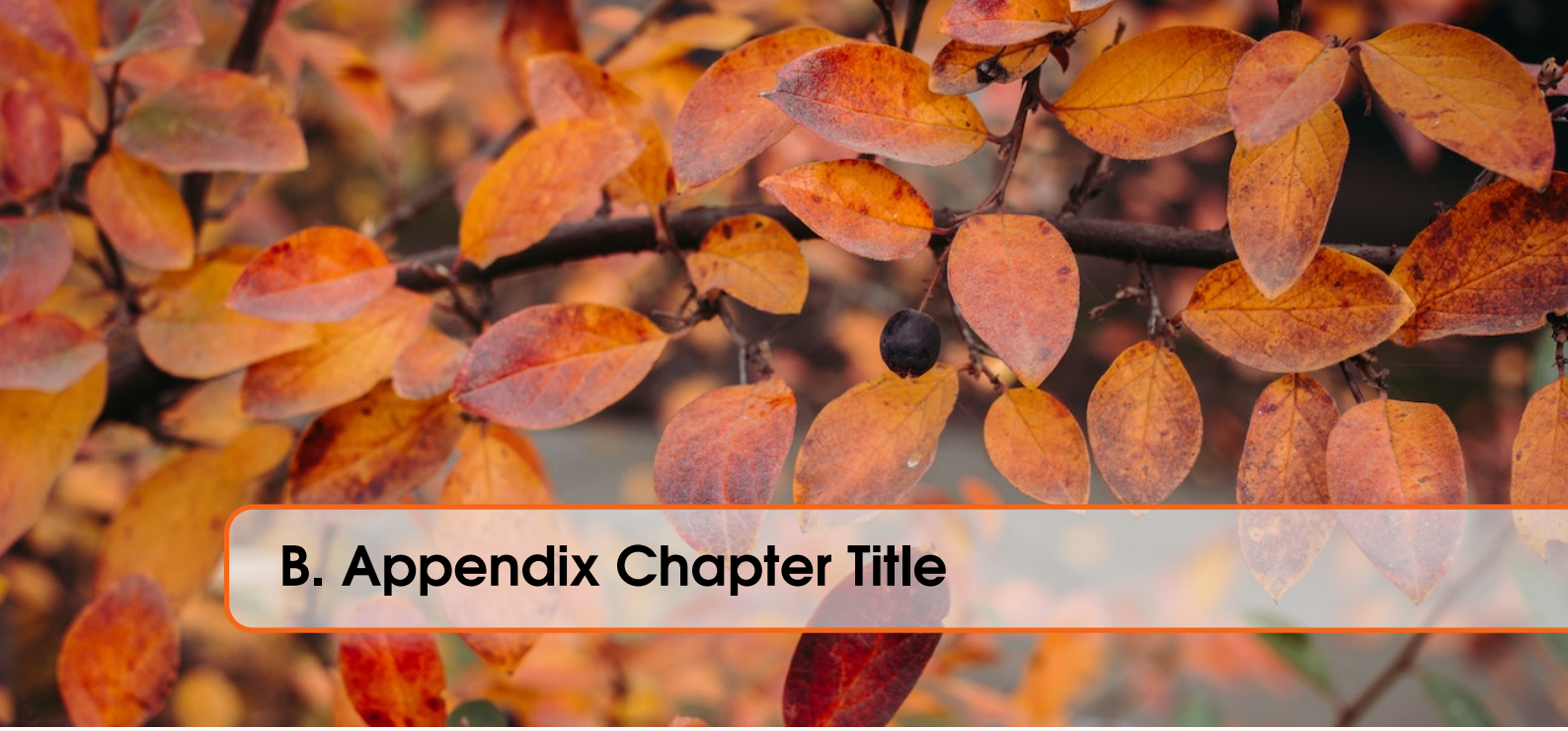
Index



A. Appendix Chapter Title

A.1 Appendix Section Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam auctor mi risus, quis tempor libero hendrerit at. Duis hendrerit placerat quam et semper. Nam ultricies metus vehicula arcu viverra, vel ullamcorper justo elementum. Pellentesque vel mi ac lectus cursus posuere et nec ex. Fusce quis mauris egestas lacus commodo venenatis. Ut at arcu lectus. Donec et urna nunc. Morbi eu nisl cursus sapien eleifend tincidunt quis quis est. Donec ut orci ex. Praesent ligula enim, ullamcorper non lorem a, ultrices volutpat dolor. Nullam at imperdiet urna. Pellentesque nec velit eget est euismod pretium.



B. Appendix Chapter Title

B.1 Appendix Section Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam auctor mi risus, quis tempor libero hendrerit at. Duis hendrerit placerat quam et semper. Nam ultricies metus vehicula arcu viverra, vel ullamcorper justo elementum. Pellentesque vel mi ac lectus cursus posuere et nec ex. Fusce quis mauris egestas lacus commodo venenatis. Ut at arcu lectus. Donec et urna nunc. Morbi eu nisl cursus sapien eleifend tincidunt quis quis est. Donec ut orci ex. Praesent ligula enim, ullamcorper non lorem a, ultrices volutpat dolor. Nullam at imperdiet urna. Pellentesque nec velit eget est euismod pretium.